

Minimum Weighted Hazard-Exposure Dispatch: Complexity, an Exact Algorithm, and an FPTAS

Quang-Vinh Dang^{1*}, Minh Ngoc Dinh², Hoang-Viet Vu¹,
Phuc-Son Nguyen³

^{1*}British University Vietnam, Hung Yen, Vietnam.

²Millennia Education, Ho Chi Minh City, Vietnam.

³UEH University, Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): vinh.dq4@buv.edu.vn;

Contributing authors: minh.dinh@maeducation.com;

30066555@st.buv.edu.vn; son.np3393@ueh.edu.vn;

Abstract

We study a dispatch-scheduling problem arising when a single depot must send a vehicle or crew to sites before a spreading hazard (flood, wildfire, contamination plume) reaches them. Each site has a round-trip dispatch time, a hazard-arrival deadline and a criticality weight; the objective is a dispatch order maximizing the total weight of sites served in time. We call this Minimum Weighted Hazard-Exposure Dispatch (MWHED) and show it is exactly the classical problem $1||\sum w_j U_j$, so its basic picture is inherited from scheduling theory: weak NP-hardness (the equal-deadline case is 0/1 knapsack), a Lawler–Moore-type pseudo-polynomial dynamic program, and a value-scaling FPTAS. We present these in a self-contained form for the transportation audience and say precisely which are classical. The framing adds a classification: restricted to the instances in which some of the dispatch times, weights and deadlines are constant, the problem is polynomial exactly when dispatch times or weights are constant and weakly NP-hard otherwise, so heterogeneous deadlines alone never cause hardness; and a matroid-greedy algorithm for equal dispatch times that extends to any number of identical vehicles. Natural heuristics (deadline order with or without skipping, a weighted greedy repair) have unbounded worst-case ratio, and the FPTAS analysis is tight up to a factor $1 - \epsilon^2$. A numerical study with active FPTAS rounding, verified against exhaustive search, and a Camp Fire case study with a sensitivity analysis complete the paper.

Keywords: combinatorial optimization, scheduling under deadlines, computational complexity, approximation algorithms, transportation networks, disaster response

1 Introduction

A recurring operational problem in infrastructure and emergency-response networks is deciding, from a single depot, in what order to dispatch a vehicle or crew to a set of sites that are each racing against a spreading hazard. A wildfire crew choosing which structures to defend before the fire front arrives, a utility company dispatching repair trucks to substations before a cascading outage propagates to them, and a flood-response team evacuating or servicing neighborhoods before rising water reaches them all face the same underlying decision: given a known rate at which a hazard is spreading, and a known cost of reaching each site from a common depot, in what order should sites be visited to save as much value as possible before the hazard reaches each one in turn?

This decision is a scheduling problem, not merely a routing one. Once every visit is a self-contained round trip from a single depot—the standard topology for a technician, repair-crew, or single-vehicle emergency-response dispatch base—the cost of visiting a site does not depend on which site was visited immediately before it, only on the cumulative time already spent. The question is then purely one of *order*: which sites to reach early, which to defer, and which to concede to the hazard, given that each site i becomes “exposed” once the cumulative dispatch time exceeds its known hazard-arrival deadline d_i , at a loss of w_i .

The three motivating settings above share a further feature that makes the *ordering* decision, rather than the underlying routing or hazard-forecasting problem, the right object of study. In each case, the hazard’s own spread is typically produced by a separate, already well-developed forecasting step—a wildfire-spread simulator driven by fuel load, wind, and terrain; a cascading-failure model for a power network; a hydrological model for a flood plain—so that, by the time a dispatcher is choosing a visiting order, each site’s arrival deadline d_i is already a given input, not something to be modeled jointly with the dispatch decision. Similarly, the round-trip dispatch time p_i is typically fixed by the depot’s location and the local road or access network, and is known before dispatch begins. What remains undetermined, and squarely a combinatorial decision, is the *order* in which a single crew or vehicle should visit sites once p_i and d_i are fixed—exactly the question MWHED isolates. This separation of concerns (forecast the hazard and fix the travel network first; optimize the dispatch order second) mirrors how the wildfire operations-research literature we discuss in Section 2 is typically organized, and is what lets us give a complete complexity and algorithmic answer to the ordering question alone, rather than a heuristic for the joint problem.

We formalize this as Minimum Weighted Hazard-Exposure Dispatch (MWHED) ~~and give a complete complexity and algorithmic picture. Our contributions are:~~

~~We show MWHED is exactly . It is worth saying at the outset, and Section 3 proves it, that MWHED is exactly the classical single-machine scheduling problem 1|| $\sum w_j U_j$ of maximizing the weighted number of on-time jobs, and prove it . Consequently the basic complexity picture (weak NP-hardness, a pseudo-polynomial dynamic program, an approximation scheme) is inherited from scheduling theory, and we do not claim a new algorithmic technique for the general problem. Our~~

contributions are therefore of two kinds. The first is a self-contained development of the classical results in the transportation vocabulary, with complete proofs and a running example, aimed at a reader who would not otherwise connect a dispatch problem to $1/\|\sum w_j U_j\|$.

1. ~~MWHED is weakly NP-hard by an elementary, self-contained argument: the special case in which every site shares the same hazard deadline is *exactly*, by a direct reduction from PARTITION that uses only the equal-deadline special case, which is *exactly* the 0/1 knapsack problem (Theorem 2).~~
2. ~~We give an An exact pseudo-polynomial dynamic program (Theorem 3), derived directly derived from an earliest-deadline-first exchange argument that we prove from first principles rather than assert (Theorem 3); this is the Lawler–Moore dynamic program (Lawler and Moore 1969).~~
3. ~~We adapt the classical A value-scaling technique to obtain a fully polynomial-time approximation scheme (FPTAS) for MWHED–FPTAS (Theorem 4), so that a solution within any desired factor $1 - \epsilon$ of optimal is computable in time polynomial in the number of sites and $1/\epsilon$; approximation schemes for this problem are classical (Sahni 1976; Gens and Levner 1981), and we present the construction together with a complete proof and an explicit treatment of the case in which the scaling is vacuous.~~

The second kind is what we believe the transportation framing adds, and where we have tried to go beyond a relabelling:

4. ~~We identify a second special case—equal round trip dispatch times, i.e., sites equidistant from the depot—that is solvable *exactly* in $O(n \log n)$ time via a matroid greedy algorithm—A *complete classification by homogeneity* (Theorem 5), showing that heterogeneity in dispatch cost *alone* does not cause hardness: it is the combination of heterogeneous dispatch cost and heterogeneous criticality, under a shared deadline structure, that does9). For every restriction obtained by requiring some of the dispatch times, weights, and deadlines to be constant, MWHED is polynomial if dispatch times or weights are constant and weakly NP-hard otherwise; heterogeneity of the hazard deadlines never causes hardness by itself. Together with the equal-cost result below this turns the informal question “which network structure makes dispatch easy” into a precise statement.~~
5. ~~We show the natural fallback for a planner who declines to optimize at all—dispatching sites in increasing deadline order and simply accepting whichever remain on time—has unbounded worst-case loss relative to W^* (Proposition 6), so Theorems 3 and 4 are not merely convenient but necessary for any *Equal dispatch times, with identical vehicles* (Theorem 5). A matroid-greedy algorithm solves the equal-cost case in $O(n \log n)$ time for one vehicle and, by the same proof, for any number m of identical vehicles. We prove the required matroid property from scratch rather than cite it. In particular, the equal-cost case does *not* become hard when several vehicles are available (it does once release dates are added; Section 6.1).~~
6. *Worst-case results for natural heuristics* (Propositions 6, 7, 8). Dispatching in deadline order, with or without skipping sites that would be late, and a weighted

greedy repair all have unbounded worst-case ~~guarantee-at-all-ratio~~; and the standard FPTAS analysis is tight up to a factor $1 - \epsilon^2$. These results say that some form of reconsideration is needed for a guarantee; they do *not* say that the dynamic program or the FPTAS is the only way to obtain one.

Together, Theorems 2 and 5 give a sharp structural picture of where the difficulty in this problem lives, Theorems 3 and 4 give planners exact and provably near-exact tools for the general, hard case, and Proposition 6 shows those tools are not optional: the obvious do-nothing alternative can be made to fail arbitrarily badly. A numerical illustration in ~~Finally~~, Section 5 confirms the FPTAS tracks the true optimum closely well before its worst-case guarantee requires, while a naive dispatch order (no optimization at all, just visiting sites in deadline order) degrades as instances grow. ~~reports a numerical study designed so that the FPTAS's rounding is verifiably active (with small weights it is vacuous, as Section 5.2 explains), together with exhaustive-search verification of every algorithm, and a real-wildfire case study with a sensitivity analysis over its uncertain inputs.~~ To keep the argument concrete ~~rather than purely symbolic~~, Sections 3 and 4 carry a single four-site running example through ~~every result~~ the results, worked out by hand alongside each proof.

The remainder of this paper is organized as follows. Section 2 positions MWHED against the scheduling, real-time-systems, and disaster-response ~~routing literatures~~ literatures and states precisely what is classical. Section 3 defines MWHED ~~formally~~ and establishes the earliest-deadline-first ~~structural~~ lemma every later result depends on. Section 4 proves the ~~four theorems and the proposition results~~ above. Section 5 reports ~~four complementary numerical experiments~~ the numerical study and the case study. Section 6 discusses ~~three natural extensions~~ extensions (several vehicles, a moving depot, and uncertain parameters) and precisely what each breaks in our analysis, and Section 7 concludes. Two appendices give an exhaustive hand-verification of the running example and a ~~fully~~ spelled-out derivation of the FPTAS ~~'s approximation-ratio~~ bound.

2 Related work

Scheduling with deadlines. MWHED, once its transportation framing is stripped away, is the classical single-machine problem of maximizing the weighted number of on-time jobs, equivalently minimizing the weighted number of late jobs, $1 \parallel \sum w_j U_j$ in the standard three-field notation (job j has processing time p_j , due date d_j and weight w_j ; $U_j = 1$ if it is late). Moore (1968) gives an $O(n \log n)$ algorithm for the *unweighted* case ($1 \parallel \sum U_j$): process jobs in earliest-due-date order, and whenever the running schedule becomes infeasible, permanently discard the scheduled job with the largest processing time ~~among those scheduled so far. This procedure is commonly called (the Moore-Hodgson algorithm-).~~ For the weighted case, Lawler and Moore (1969) give a ~~general functional-equation (dynamic programming) technique for resource allocation and sequencing problems that specializes to a pseudo-polynomial algorithm for the weighted case; the classification of $1 \parallel \sum w_j U_j$ itself as (weakly) dynamic program over jobs in due-date order and total processing time that runs in $O(nP)$ time, $P = \sum_j p_j$; this is our Theorem 3. The problem is weakly NP-hard~~ is standard

in the scheduling-complexity literature, surveyed by Lenstra et al. (1977), and follows directly from the same reduction we give in Section 4: since an instance with a common due date is exactly a 0/1 knapsack instance, whose NP-hardness is classical (Garey and Johnson 1979). We are not aware of a prior statement of this reduction, or of complexity surveys include Lenstra et al. (1977). Approximation schemes are also classical: Sahni (1976) gives rounding-based approximation algorithms for single-processor job sequencing with deadlines, which is the same problem, and Gens and Levner (1981) give a faster approximation algorithm for it. Our Theorem 4 is therefore a re-derivation of a known type of result in our notation, not a new technique, and the same holds for the common-deadline hardness argument and the equal-processing-time tractable special case of Theorem 5, ~~in transportation network language~~ which is the classical unit-time job sequencing with profits. What we are not aware of, and what Sections 4–5 add, is the classification of Theorem 9, the extension to identical vehicles of Theorem 5, the heuristic lower bounds of Propositions 6–8, and the verified computational study.

This classical scheduling problem remains an active research topic: Hejl et al. (2022) give a refined pseudo-polynomial dynamic program for the practically important “strongly correlated” regime (where processing times and weights are close in value), and Hermelin et al. (2024) improve the Lawler–Moore-style DP’s running time via $(\max, +)$ -convolution techniques—an algorithmic refinement orthogonal to, and compatible with, Theorem 3’s DP. Heeger and Hermelin (2024) sharpen the ~~complexity picture~~ further parameterized picture, proving $1 \parallel \sum w_j U_j$ is $W[1]$ -hard parameterized by ~~either the number of distinct processing times or~~ and also by the number of distinct weights in the instance—a result that, translated. Translated into MWHED’s language, says that even a small: there is, unless $FPT = W[1]$, no algorithm running in $f(k) \cdot \text{poly}(n)$ time where k is the number of distinct dispatch costs times (or criticality levels ~~does not, by itself, guarantee tractability, sharpening the boundary~~ Theorem 5 and Remark 4 draw between the fully-heterogeneous and equal-cost cases), so a small number of distinct values does not make the problem easy in the fixed-parameter sense. For a fixed k the problem remains polynomial (a dynamic program over the counts of selected sites of each type has $n^{O(k)}$ states) and for $k = 1$ it is the tractable case of Theorem 5 (or Moore–Hodgson, for equal weights), so these results are consistent with, and refine, the boundary that Theorem 9 draws. On the heuristic side, Antonov et al. (2026) give a data-driven (machine-learning-guided) heuristic for the same objective, a more sophisticated relative of the weighted greedy-repair heuristic we use as an empirical baseline in Section 5.

Real-time and deadline-driven systems. Earliest-deadline-first ordering is also the central object of classical real-time scheduling theory, where Liu and Layland (1973) show that, for periodic tasks on a single processor, earliest-deadline-first achieves full processor utilization while any fixed-priority policy (such as rate-monotonic scheduling) cannot. That literature’s EDF result and our Lemma 1 are structurally analogous—both establish that ordering by deadline is necessary and sufficient for a natural notion of feasibility—but the objectives differ fundamentally: real-time scheduling asks whether *every* task can be met, a feasibility question with

a clean necessary-and-sufficient utilization test, whereas MWHED presupposes infeasibility is unavoidable when dispatch costs are large relative to deadlines, and asks instead which *weighted subset* of sites to sacrifice—the optimization question that drives Theorem 2’s hardness result and has no analogue in the feasibility-testing literature. It is worth noting why the two literatures’ core feasibility results still agree despite this difference in objective: Liu and Layland’s utilization test is a necessary-and-sufficient *aggregate* condition (a single inequality over all tasks) precisely because their tasks recur periodically forever, so any transient slack evens out over a long enough horizon; MWHED’s sites are each served at most once, so no such aggregate test can substitute for Lemma 1’s site-by-site exchange argument, and no analogous single-inequality feasibility test exists for MWHED’s one-shot dispatch problem—indeed, Theorem 2 shows that even deciding the size of the best *weighted* feasible subset, let alone a simple aggregate test for it, is NP-hard. More recent real-time scheduling work extends EDF-style analysis to richer task models: Günzel et al. (2022) give a unified schedulability test for self-suspending periodic tasks that recovers EDF (among other policies) as a special case, and Wang et al. (2025) refine the schedulability analysis for that same self-suspending, EDF-like setting. Both retain the periodic, run-forever structure that makes an aggregate utilization test possible; MWHED’s one-shot, single-visit-per-site structure is exactly what rules out an analogous test here.

Approximation. The 0/1 knapsack problem admits a fully polynomial-time approximation scheme via scaling and rounding of item values, due to Ibarra and Kim (1975); ~~we adapt this technique directly to MWHED’s dynamic program in the same technique applies to job sequencing with deadlines (Sahni 1976; Gens and Levner 1981), and Theorem 4. The adaptation is not entirely mechanical: knapsack’s FPTAS scales values and reuses a single time-indexed capacity constraint, whereas MWHED has is this construction written for MWHED. The one point at which the presentation differs from knapsack is that there is no single shared capacity—each site carries its own deadline—so capacity (each site has its own deadline), so the value-indexed dual table of Theorem 4’s proof must track table must test feasibility against each site’s individual d_i rather than a single global bound; this is easy in deadline order, and the case split we make explicit in Appendix B (between the scaling being vacuous or not) has no counterpart in the standard knapsack presentation, where $w_{\max}$ and n play a comparatively simpler role whether the scaling is vacuous) is simply careful bookkeeping, not a new idea.~~ The classical $O(n^3/\epsilon)$ -style ~~FPTAS we adapt scheme~~ is not the fastest knapsack FPTAS known today: Jin (2019) improve the running time via a more careful scaling scheme, ~~known even for knapsack; Jin (2019) and Chen et al. (2025) give a nearly-quadratic-time FPTAS, $\tilde{O}(n + 1/\epsilon^2)$, that is conditionally near-optimal. We adapt the classical (rather than the fastest known) faster schemes, and Gens and Levner (1981) give a faster scheme for job sequencing with deadlines itself. We keep the classical construction because its ease structure is what Theorem 4’s proof and Appendix B’s derivation build on directly; porting the faster constructions to MWHED’s per-site-deadline setting (rather than knapsack’s single shared capacity) is a natural avenue for tightening Theorem 4’s~~

~~runtime bound~~ analysis is the cleanest and because Proposition 8 shows that analysis is tight.

Matroids and greedy algorithms. The equal-processing-time special case of Theorem 5 is an instance of the classical problem of scheduling unit-time jobs with deadlines to maximize total profit, whose feasible job sets form a matroid, on which a greedy algorithm is provably optimal; we follow the treatment in Korte and Vygen (2018). This matroid structure is exactly what breaks once dispatch costs become heterogeneous, which is why Theorem 5’s greedy algorithm has no counterpart for general MWHED. Concretely, in the equal-deadline restriction (which Theorem 2 identifies with 0/1 knapsack of capacity D), take three sites with dispatch times 2, 1, 5 and a common deadline $D = 5$: $\{3\}$ (the size-5 site alone) is feasible and already uses the entire capacity, while $\{1, 2\}$ (total size 3) is also feasible; but neither remaining site can be added to $\{3\}$ ($5 + 2$ and $5 + 1$ both exceed $D = 5$), so a feasible set of size 1 fails to extend toward a feasible set of size 2 by any element of the latter—precisely the matroid exchange property failing. This is why a single large, low-value commitment can block a strictly better pair of smaller commitments, and is the structural reason Theorem 2’s hardness survives even though the matroid case (Theorem 5) is easy. Matroid algorithms themselves remain an active area: Terao (2025) give faster matroid partition algorithms, and Ma et al. (2026) prove a greedy algorithm is simultaneously optimal and fair for a multiagent matroid-upgrading problem, reinforcing that “greedy is optimal on a matroid” continues to be a productive template well beyond the classical setting Theorem 5 uses.

Routing and dispatch under a spreading hazard. Wildfire suppression is the application area where the “race a spreading hazard” structure has been studied most directly in the operations research literature, and it illustrates the point at which our abstraction departs from that literature’s usual level of detail. Harris et al. (2023) model fire spread via a minimum-travel-time principle—the fire’s arrival time at any point is governed by the fastest path a flame front can take through spatially varying fuel and terrain, akin to a shortest-path computation over a continuous landscape—and decompose the resulting joint suppression-and-routing decision via logic-based Benders decomposition, trading exact fire-spread geometry against tractable large-scale optimization. Avci et al. (2024) extend this style of problem to multiple heterogeneous resource types (e.g., ground crews and aircraft with different suppression capabilities and travel speeds), which multiplies the routing decision’s combinatorial complexity without changing its underlying deadline structure. Alvelos (2025) go further, jointly optimizing dispatching, the positioning of resources in anticipation of the fire’s advance, and their routing, in a single integrated model. More recently, Granda et al. (2025) give a mathematical programming formulation for wildfire suppression resource allocation, and, closest in spirit to our own complexity results, Delazeri and Ritt (2026) prove NP-completeness for a landscape-graph model of wildfire suppression resource allocation, alongside a competitive mixed-integer program and a physics-grounded (Rothermel fire-spread model) instance generator—an independent hardness result in essentially our application domain, though for a differently structured (graph-based, simultaneous multi-resource) model than MWHED’s single-vehicle dispatch-ordering question. Two recent systematic reviews position the broader

literature: Tezcan and Eren (2025) survey optimization approaches to forest-fire suppression, and Rostamian et al. (2026) review 177 papers on wildfire management optimization under uncertainty and complexity, both confirming that pure complexity-theoretic characterizations of a dispatch-ordering subproblem—as opposed to joint suppression, positioning, and routing models—remain rare in this literature. Across all of these, the fire’s own spatial spread is modeled in detail and coupled tightly to the routing and positioning decisions; MWHED instead abstracts the hazard’s spread into a per-site deadline (a natural reduction once the hazard’s spread model is fixed and known, as is standard once a fire, flood, or contamination-plume simulator has produced arrival-time estimates for each site) and asks the sharper question of what complexity and algorithmic guarantees are available for the resulting pure dispatch-ordering decision. To our knowledge, no prior work in this literature establishes the knapsack-equivalence hardness result, the exact dynamic program, the FPTAS, or the equal-cost tractable case we give here.

Positioning. MWHED ~~sits at the intersection of these four literatures without duplicating any of them: it inherits its complexity classification from classical scheduling theory, both the founding results and their recent refinements (Moore 1968; Lawler and Moore 1969; Lenstra et al. 1977; Hejl et al. 2022; Hermelin et al. 2024; Heeger and scheduling theory (Moore 1968; Lawler and Moore 1969; Lenstra et al. 1977; Sahni 1976; Gens and Levner 1997), its algorithmic toolkit from knapsack approximation and matroid theory, including recent speedups (Ibarra and Kim 1975; Korte and Vygen 2018; Jin 2019; Chen et al. 2025; Terao 2025; Ma et al. 2026), its deadline-ordering intuition from real-time systems (Liu and Layland 1973; Günzel et al. 2022; Wang et al. 2025), and its motivating application-motivation from disaster-response operations research (Harris et al. 2023; Avci et al. 2024; Alvelos 2025; Granda et al. 2025; Delazeri and Ritt 2026; Tezcan and Eren 2025; Rostamian et al. 2026), but the specific problem—and every result we prove about it—is new.~~ Its theoretical results are, with the exceptions listed above, classical ones restated in transportation language; Table 2 in Section 4 summarizes how each of our five main results maps onto this classification. The overall marks which. We believe the pattern—a transportation reparametrization of a classical scheduling problem, ~~whose hardness, exact algorithm, and approximation scheme all follow by directly adapting standard techniques, while a natural structural restriction (equal dispatch cost) recovers tractability—is, we believe, a template that could apply to other transportation-network problems with a similar whose tractability boundary has a network interpretation—applies to other~~ deadline-driven ~~structure, not only to the specific hazard-dispatch motivation we use here~~ transportation problems; Section 6 discusses ~~three such extensions~~ concretely extensions.

3 Model

Definition 1 (Minimum Weighted Hazard-Exposure Dispatch) An instance of MWHED consists of n sites $1, \dots, n$ and a depot. Each site i has a round-trip dispatch time $p_i \in \mathbb{Z}_{>0}$ (the time to travel from the depot to site i and back, or more generally the time the depot’s single vehicle or crew is occupied while serving site i), a hazard-arrival deadline $d_i \in \mathbb{Z}_{>0}$ (the known time at which a spreading hazard reaches site i), and a criticality weight $w_i \in \mathbb{Z}_{>0}$. A *dispatch order* is a permutation σ of $\{1, \dots, n\}$; the *completion time* of site $\sigma(k)$ is

$C_{\sigma(k)}(\sigma) := \sum_{j=1}^k p_{\sigma(j)}$, and site i is *on time* under σ if $C_i(\sigma) \leq d_i$, otherwise *exposed*. The objective is to choose σ maximizing the total on-time weight

$$W(\sigma) := \sum_{i=1}^n w_i \cdot \mathbb{1}[C_i(\sigma) \leq d_i]. \quad (1)$$

Table 1 collects the notation introduced in this definition and used throughout the rest of the paper, for reference.

Table 1 Notation used throughout the paper.

Symbol	Meaning
n	number of sites
p_i	round-trip dispatch time for site i
d_i	hazard-arrival deadline for site i
w_i	criticality weight for site i
σ	a dispatch order (permutation of sites)
$C_i(\sigma)$	completion time of site i under order σ
$W(\sigma)$	total on-time weight of order σ , Eq. (1)
W^*	the optimal (maximum) value of $W(\sigma)$
S	a subset of sites dispatched on time
P	$\sum_{i=1}^n p_i$, the total dispatch time (Theorem 3)
$f(i, t)$	exact DP table: max weight using sites $1..i$, time exactly t
ϵ	FPTAS accuracy parameter, $\epsilon \in (0, 1)$ (Theorem 4)
K	FPTAS weight-scaling factor
w'_i	scaled weight, $\lfloor w_i/K \rfloor$
$g(i, v)$	FPTAS's value-indexed dual DP table
D_i	latest feasible dispatch position, equal-cost case, $\lfloor d_i/p \rfloor$
<u>m</u>	<u>number of identical vehicles (MWHED-m, Section 4.4)</u>

Because every visit returns to the depot before the next dispatch begins, p_i does not depend on which site precedes or follows site i in the order—the defining structural feature that distinguishes MWHED from a routing problem and places it in the scheduling family. Minimizing total *exposure cost* $\sum_i w_i(1 - \mathbb{1}[C_i(\sigma) \leq d_i])$ is equivalent to maximizing $W(\sigma)$, since $\sum_i w_i$ is a constant of the instance; we work with the maximization form throughout.

Assumption 1 (Individually feasible sites) Every site satisfies $p_i \leq d_i$.

This is without loss of generality: a site with $p_i > d_i$ cannot be on time under any dispatch order, even served first, so it can be deleted from the instance in a preprocessing pass without changing the optimal value achievable on the remaining sites. One immediate consequence we use later: the single highest-weight site, dispatched alone, is itself a feasible subset, so $W^* \geq w_{\max} := \max_i w_i$.

Remark 1 (Return and arrival readings of the deadline) [Definition 1](#) counts a site as on time when the vehicle's completion of the whole p_i -long occupancy—the round trip, including the return—happens by d_i (the return reading). When what matters is that the vehicle arrive at the site before the hazard does, only the outbound part $a_i \leq p_i$ of the occupancy must fit: site i is protected iff $C_i - p_i + a_i \leq d_i^{\text{haz}}$ where d_i^{haz} is the hazard-arrival time. This is the same test with deadline $d_i := d_i^{\text{haz}} + (p_i - a_i)$, which for a symmetric trip is $d_i^{\text{haz}} + p_i/2$. Hence the arrival reading is an instance of [Definition 1](#) with shifted deadlines, and every result below applies to it unchanged; only the input d_i differs. The two readings can give different answers on the same data, so a practitioner must state which one the hazard-arrival times refer to; [Section 5.6](#) uses the arrival reading.

The following lemma is the structural fact both the dynamic program of [Section 4](#) and the tractability results depend on. It is the transportation-scheduling analogue of the classical fact that earliest-due-date order minimizes maximum lateness on a single machine.

Lemma 1 (Earliest-deadline-first feasibility) *Fix a subset $S \subseteq \{1, \dots, n\}$. There exists a dispatch order under which every site in S is on time if and only if, when the sites of S are dispatched first (before any site outside S) in order of increasing deadline d_i , every site in S is on time.*

Proof Sufficiency is immediate: the stated order is one such dispatch order. For necessity, suppose S is feasible under some order σ (every site of S on time). We transform σ into the stated order by two kinds of exchanges, each of which preserves feasibility of S .

First, suppose some site $j \notin S$ precedes some site $i \in S$ in σ . Moving j to immediately after the last element of S (and shifting the intervening sites earlier by p_j) strictly decreases the completion time of every site of S that came after j , and leaves the completion time of every site of S that came before j unchanged; it therefore preserves feasibility of every site in S , and after repeating this for every $j \notin S$ that precedes some element of S , all of S forms a prefix of the schedule.

Second, with S now a prefix, suppose two adjacent sites $i, i' \in S$ appear in the order with i immediately before i' , yet $d_i > d_{i'}$ —the later-deadline site is scheduled first. Let t_0 be the cumulative dispatch time immediately before this pair is served (unaffected by how the pair is internally ordered, since it depends only on the sites preceding them). Under the current order, i completes at $t_0 + p_i$ and i' completes at $t_0 + p_i + p_{i'}$; since the order is feasible for S , in particular $t_0 + p_i + p_{i'} \leq d_{i'}$. Swap the pair so that i' precedes i : every other site in S still finishes at the same time as before, i' now completes at $t_0 + p_{i'}$, and i now completes at $t_0 + p_{i'} + p_i$ —the same total as before, since the pair's combined processing time is unchanged by reordering it. Because $p_i > 0$, $t_0 + p_{i'} < t_0 + p_i + p_{i'} \leq d_{i'}$, so i' remains on time; and $t_0 + p_i + p_{i'} \leq d_{i'} < d_i$, so i remains on time as well. Repeating this adjacent-transposition argument (a bubble sort on the prefix) sorts S into increasing- d_i order without ever violating a previously-satisfied deadline. $\square$

[Lemma 1](#) lets us restrict attention, without loss of generality, to dispatch orders that serve some subset S first, in increasing-deadline order, followed by the remaining

(necessarily exposed) sites in any order; the optimization problem becomes that of choosing S to maximize $\sum_{i \in S} w_i$ subject to the feasibility condition of Lemma 1.

Example 1 (A running illustration) Throughout this section we illustrate each result on a single small instance: four sites with round-trip dispatch times $p = (2, 3, 1, 4)$, hazard deadlines $d = (4, 6, 2, 9)$, and criticality weights $w = (5, 8, 3, 10)$, all satisfying Assumption 1. Sorting by increasing deadline gives the order 3, 1, 2, 4 (deadlines 2, 4, 6, 9); Lemma 1 says we may restrict to dispatch orders that serve some subset S first, in exactly this relative order.

Dispatching every site in this order—the naive policy of simply visiting sites earliest-deadline-first, with no further optimization—gives completion times 1, 3, 6, 10 for sites 3, 1, 2, 4 respectively; the first three are on time, but site 4, with deadline 9, arrives at cumulative time 10 and is exposed, losing its full weight $w_4 = 10$. This naive order captures only $w_3 + w_1 + w_2 = 3 + 5 + 8 = 16$ of the total weight 26.

The optimal choice of S is instead $\{1, 2, 4\}$, excluding site 3: dispatched in deadline order 1, 2, 4, completion times are 2, 5, 9, each on time, for a total on-time weight of $W^* = 5 + 8 + 10 = 23$ —strictly better than the naive 16, by deliberately sacrificing site 3 (deadline 2, weight only 3) to make room for the far more valuable site 4. No dispatch order can do better, as Example 3’s dynamic program table confirms exhaustively. This gap between naive earliest-deadline dispatch and the true optimum is ~~exactly the phenomenon what~~ Section 5’s numerical study documents at scale measures on random instances.

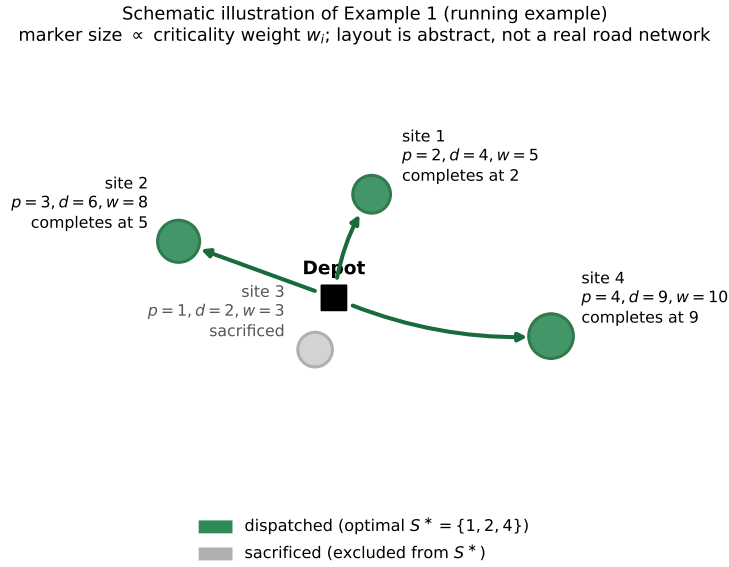


Fig. 1 A schematic illustration of Example 1. Site positions are laid out at a radius proportional to each site’s round-trip dispatch time p_i , purely for visual spacing—this is an abstract schematic of the instance, not a real road network or a geographic map (MWHED’s theory is instance-agnostic and this paper uses no real-world coordinate data). Marker size is proportional to criticality weight w_i ; green markers and arrows show the optimal dispatch $S^* = \{1, 2, 4\}$ and each site’s completion time; the gray marker is site 3, sacrificed to make room for the higher-value site 4.

4 Complexity and algorithms

Table 2 previews ~~how the five the~~ results of this section ~~fit together, before we state and prove each in turn~~ and marks which are classical (stated here in transportation language, with complete proofs) and which we believe are new.

Table 2 Summary of this section’s results. n is the number of sites, $P = \sum_i p_i$, and $\epsilon \in (0, 1)$ is the FPTAS’s accuracy parameter.

Setting	Status	Result	Runtime	<u>Origin</u>
General instance	weakly NP-hard	Thm. 2	—	<u>classical</u>
General instance	solved exactly	Thm. 3	$O(nP)$	<u>Lawler–Moore</u>
General instance	(1 ϵ) -approximated	Thm. 4	$O(n^3/\epsilon)$	<u>classical type</u>
Equal deadlines ($d_i=D$)	weakly NP-hard (= knapsack)	Thm. 2	—	<u>classical</u>
Equal cost ($p_i=p$ weights)	<u>solved exactly</u>	<u>Thm. 9(a)</u>	<u>$O(n \log n)$</u>	<u>Moore</u>
<u>Equal cost, $m = 1$</u>	solved exactly	Thm. 5	$O(n \log n)$	<u>classical</u>
<u>Equal cost, $m > 2$ identical vehicles</u>	<u>solved exactly</u>	<u>Thm. 5</u>	<u>$O(n \log n)$</u>	<u>new (proof)</u>
<u>Which of p, w, d are constant</u>	<u>polynomial iff p or w constant</u>	<u>Thm. 9</u>	<u>—</u>	<u>new framing</u>
Naive EDD, no repair EDD with skipping	unbounded worst-case ratio	Prop. 6	$O(n \log n)$	<u>new</u>
<u>Weighted greedy repair</u>	<u>unbounded ratio</u>	<u>Prop. 7</u>	<u>—</u>	<u>new</u>
<u>FPTAS analysis</u>	<u>tight to $1 - \epsilon^2$</u>	<u>Prop. 8</u>	<u>—</u>	<u>new</u>

4.1 Hardness

We first isolate the one fact about completion times on which the hardness argument rests. Recall that, by the defining property of MWHED, the occupancy p_i of the vehicle at site i does not depend on which site was served before it.

Fact 1 (Feasibility under a common deadline) *Suppose $d_i = D$ for every site, and let $S \subseteq \{1, \dots, n\}$. There is a dispatch order under which every site of S is on time if and only if $\sum_{i \in S} p_i \leq D$.*

Proof By Lemma 1 we may serve the sites of S first, in any order (all deadlines are equal, so the earliest-deadline-first order is arbitrary among them). The vehicle is never idle and the occupancies are fixed numbers, so the k -th site served completes at the partial sum of the first k occupancies, and the last site of S completes at the sum $T(S) := \sum_{i \in S} p_i$ of all of their occupancies, whatever the order. This is the only additivity used: it is a statement about the completion time of the last site, not about every site's completion time being additive.

($\Rightarrow$) If every site of S is on time, then in particular the last one is, so $T(S) \leq D$. ($\Leftarrow$) If $T(S) \leq D$, then every site of S completes at a partial sum of positive numbers adding up to $T(S)$, hence at some time $\leq T(S) \leq D$, so every site is on time. $\square$

Theorem 2 (NP-hardness of MWHED) *MWHED is NP-hard. This holds even ~~restricted to instances in which~~ when every site shares a common hazard deadline, $d_i = D$ for all i , and even when $w_i = p_i$ for all i . Precisely, the decision problem “is $W^* > k$?” is NP-complete.*

Proof ~~Consider the restriction $d_i = D$.~~ Membership in NP is clear: a dispatch order is a certificate and $W(\sigma)$ is computed in $O(n)$ time. For hardness we reduce from PARTITION (Garey and Johnson 1979): given positive integers $a_1, \dots, a_n$ with $A := \sum_i a_i$ even, decide whether some $S \subseteq \{1, \dots, n\}$ has $\sum_{i \in S} a_i = A/2$. If some $a_i > A/2$ the answer is no and we output a fixed no-instance, so assume $a_i \leq A/2$ for all i . ~~By Lemma 1, a subset~~ Build the MWHED instance with

$$p_i = w_i = a_i, \quad d_i = D := A/2 \quad (i = 1, \dots, n),$$

which satisfies Assumption 1 and is built in polynomial time. By Fact 1 a set S ~~is feasible~~ can be served entirely on time iff $\sum_{i \in S} a_i \leq A/2$ and then its on-time weight is $\sum_{i \in S} w_i = \sum_{i \in S} a_i$. Since only the sites of a feasible set can be on time, $W^* = \max\{\sum_{i \in S} a_i : \sum_{i \in S} a_i \leq A/2\} \leq A/2$, with equality if and only if ~~dispatched in any order~~ (deadlines being tied, the earliest-deadline-first condition is vacuous), every site in S completes by D ; since completion times along a fixed set are additive, this holds exactly when $\sum_{i \in S} p_i \leq D$. The optimization problem

$$\max_{S \subseteq \{1, \dots, n\}} \sum_{i \in S} w_i \quad \text{s.t.} \quad \sum_{i \in S} p_i \leq D$$

~~is precisely~~ some subset sums to exactly $A/2$. Thus $W^* > A/2$ holds iff the PARTITION instance is a yes-instance. $\square$

With general weights the common-deadline restriction is exactly the 0/1 knapsack problem, with p_i the item sizes, w_i the item values, and D the capacity. The (decision version of the) 0/1 knapsack problem is NP-complete (Garey and Johnson 1979), by a polynomial reduction from PARTITION. Hence MWHED restricted to a common deadline is NP-hard, and since this is a special case of the general problem, MWHED itself is NP-hard.

~~Theorem 2's reduction is deliberately the simplest one available~~ with sizes p_i , values w_i and capacity D , by Fact 1; this is why the equal-deadline case is the natural source of hardness, and why the dynamic program of the next subsection (which is pseudo-polynomial) shows that the hardness is only *weak*. The reduction deliberately uses the simplest possible structure: it isolates ~~exactly the source of hardness, namely~~ heterogeneous dispatch costs weighed against heterogeneous criticality under a shared time budget, ~~and reduces the question of what other structure might rescue tractability to what Theorem 5 answers~~ as the source of difficulty, and Theorem 9 shows that this is the only one.

Example 2 (The running example under a common deadline) Suppose the four sites of Example 1 were instead all assigned the same hazard deadline $D = 6$ (same dispatch times and weights). By ~~Theorem 2's reduction~~ Fact 1 this is exactly a 0/1 knapsack instance with capacity 6, item sizes $p = (2, 3, 1, 4)$, and item values $w = (5, 8, 3, 10)$. Checking every subset whose total dispatch time is at most 6, the best is $\{1, 2, 3\}$, costing $2 + 3 + 1 = 6$ and worth $5 + 8 + 3 = 16$; keeping the single most valuable site instead does worse within the same budget— $\{1, 4\}$ costs 6 but is worth only 15, and $\{3, 4\}$ costs 5 but is worth only 13. Which sites to sacrifice to make room, by weight against size, cannot be read off from either quantity alone; this trade-off is exactly the source of MWHED's hardness.

4.2 An exact pseudo-polynomial algorithm

Theorem 3 (Exact dynamic program) *MWHED can be solved exactly in $O(n \cdot P)$ time and space, where $P := \sum_{i=1}^n p_i$.*

Proof Relabel sites so that $d_1 \leq d_2 \leq \dots \leq d_n$ (Lemma 1 justifies working in this order). Define, for $i = 0, \dots, n$ and $t = 0, \dots, P$,

$f(i, t) :=$ the maximum value of $\sum_{j \in S} w_j$ over $S \subseteq \{1, \dots, i\}$ with $\sum_{j \in S} p_j = t$ and S feasible.

(feasible meaning: dispatching S in increasing-deadline order, every member is on time). Set $f(0, 0) = 0$ and $f(0, t) = -\infty$ for $t > 0$. For $i \geq 1$,

$$f(i, t) = \max\left(f(i-1, t), \left[t \leq d_i\right] \cdot (f(i-1, t-p_i) + w_i)\right), \quad (2)$$

where the second term is taken as $-\infty$ if $t < p_i$ or $t > d_i$.

We verify (2) by induction on i . The first term corresponds to excluding site i from S . For the second term, suppose $S' \subseteq \{1, \dots, i-1\}$ achieves $f(i-1, t-p_i)$ and is feasible; appending site i after all of S' (site i has the largest deadline among $\{1, \dots, i\}$, so this is consistent with increasing-deadline order) does not change the completion time of any member of S' , so S' 's feasibility is preserved, and site i 's own completion time is exactly $t = (t-p_i) + p_i$, on time precisely when $t \leq d_i$. Conversely, any feasible $S \subseteq \{1, \dots, i\}$ with $i \in S$ and total time t restricts, by the same argument, to a feasible $S \setminus \{i\} \subseteq \{1, \dots, i-1\}$ with total time $t-p_i$ achieving at least $f(i, t) - w_i$ at index $i-1$; hence $f(i-1, t-p_i) \geq f(i, t) - w_i$, giving the reverse inequality and confirming (2) computes exactly $f(i, t)$.

The optimal value is $\max_{t=0}^P f(n, t)$, and the achieving subset is recovered by standard DP backtracking. The table has $O(n \cdot P)$ entries, each computed in $O(1)$ time from (2), giving $O(nP)$ total time and space. $\square$

Theorem 3 is pseudo-polynomial, not polynomial: P can be exponential in the input's bit length. This is consistent with Theorem 2's ~~reduction from knapsack, which is itself, whose hard instances need large numbers (the problem is~~ only weakly NP-hard).

Remark 2 (Relation to Lawler–Moore) ~~This is the Lawler–Moore dynamic program (Lawler and Moore 1969) for $1\|\sum w_i U_i$: jobs in due-date order, state equal to the total processing time of the jobs accepted so far, and the single test $t \leq d_i$ for an accepted job. We do not claim it as new. What the proof above adds is a self-contained derivation from the earliest-deadline-first lemma in the notation of the dispatch problem. Refinements of the $O(nP)$ bound exist (see Section 2); they apply verbatim to MWHEd.~~

Example 3 (The dynamic program on the running instance) Table 3 traces $f(i, t)$ for Example 1, relabeled in increasing-deadline order 3, 1, 2, 4 so that index $i = 1$ is site 3, $i = 2$ is site 1, $i = 3$ is site 2, and $i = 4$ is site 4, matching the order Lemma 1 fixes. Reading the last row, the largest finite entry is $f(4, 9) = 23$; backtracking (2) from this cell recovers exactly $S^* = \{1, 2, 4\}$ in the original site labels, matching Example 1. A dash marks ~~$t > d_i$ for the site just added at row i : once the cumulative time needed to include that site would already exceed $-\infty$: either the time t cannot be reached by any feasible subset of the first i sites (for instance $f(1, 2)$: site 3 alone costs 1), or including site i at that column would violate its own deadline $t \leq d_i$ (for instance $f(4, 10)$, since $d_4 = 9$); in the latter case the recursion can only carry the previous row's value forward, never improve on it, at that column and every column beyond it in the same row.~~

Table 3 The dynamic program's table $f(i, t)$ for Example 1
($i = 1$: site 3, $p = 1, d = 2, w = 3$; $i = 2$: site 1, $p = 2, d = 4, w = 5$; $i = 3$: site 2, $p = 3, d = 6, w = 8$; $i = 4$: site 4, $p = 4, d = 9, w = 10$). A dash marks $-\infty$ (infeasible or unreachable at that cell).

t	0	1	2	3	4	5	6	7	8	9	10
$f(0, t)$	0	–	–	–	–	–	–	–	–	–	–
$f(1, t)$	0	3	–	–	–	–	–	–	–	–	–
$f(2, t)$	0	3	5	8	–	–	–	–	–	–	–
$f(3, t)$	0	3	5	8	11	13	16	–	–	–	–
$f(4, t)$	0	3	5	8	11	13	16	18	21	23	–

Algorithm 1 states Theorem 3's construction explicitly, including the backtracking step used to recover an achieving subset S^* , not merely its value.

4.3 A fully polynomial-time approximation scheme

Theorem 4 (FPTAS) *For every $\epsilon \in (0, 1)$, a dispatch order σ with $W(\sigma) \geq (1 - \epsilon) \cdot W^*$ (where W^* is the true optimum) can be computed in time polynomial in n and $1/\epsilon$.*

Algorithm 1 Exact dynamic program for MWHED (Theorem 3)

Require: sites $1, \dots, n$ with $d_1 \leq d_2 \leq \dots \leq d_n$ (reabeled per Lemma 1), with dispatch times p_i , deadlines d_i , weights w_i

Ensure: optimal on-time weight W^* and an achieving subset S^*

- 1: delete every site with $p_i > d_i$ (Assumption 1); relabel the rest so $d_1 < \dots < d_n$ (Lemma 1)
- 2: $P \leftarrow \sum_{i=1}^n p_i$
- 3: $f[0][0] \leftarrow 0$; $f[0][t] \leftarrow -\infty$ for $t = 1, \dots, P$
- 4: **for** $i = 1, \dots, n$ **do**
- 5: **for** $t = 0, \dots, P$ **do**
- 6: $f[i][t] \leftarrow f[i-1][t]$ $\triangleright$ exclude site i
- 7: **if** $t \geq p_i$ **and** $t \leq d_i$ **and** $f[i-1][t-p_i] > -\infty$ **then**
- 8: **if** $f[i-1][t-p_i] + w_i > f[i][t]$ **then**
- 9: $f[i][t] \leftarrow f[i-1][t-p_i] + w_i$
- 10: $choice[i][t] \leftarrow \text{TRUE}$ $\triangleright$ site i included to reach this value
- 11: **end if**
- 12: **end if**
- 13: **end for**
- 14: **end for**
- 15: $t^* \leftarrow \arg \max_{t \in \{0, \dots, P\}} f[n][t]$; $W^* \leftarrow f[n][t^*]$
- 16: $S^* \leftarrow \emptyset$; $t \leftarrow t^*$
- 17: **for** $i = n, \dots, 1$ **do** $\triangleright$ backtrack
- 18: **if** $choice[i][t]$ **then**
- 19: $S^* \leftarrow S^* \cup \{i\}$; $t \leftarrow t - p_i$
- 20: **end if**
- 21: **end for**
- 22: **return** W^*, S^*

Proof We adapt the standard value-scaling technique used for the 0/1 knapsack FPTAS (Ibarra and Kim 1975) to the dynamic program of Theorem 3. Fix $\epsilon \in (0, 1)$ and let $w_{\max} := \max_i w_i$. Define the scaling factor $K := \max(1, \epsilon w_{\max}/n)$ and scaled weights $w'_i := \lfloor w_i/K \rfloor$, so that $w'_i \leq n/\epsilon$ for every i and hence $\sum_i w'_i \leq n^2/\epsilon$.

Run the dynamic program of Theorem 3, but in its *value-indexed* dual form: for $i = 0, \dots, n$ and $v = 0, \dots, \lfloor n^2/\epsilon \rfloor$, let

$$g(i, v) := \text{the minimum total dispatch time } \sum_{j \in S} p_j \text{ over feasible } S \subseteq \{1, \dots, i\} \text{ with } \sum_{j \in S} w'_j = v,$$

$$\underbrace{g(i, v) := \text{the minimum total dispatch time } \sum_{j \in S} p_j}_{\text{over feasible } S \subseteq \{1, \dots, i\} \text{ with } \sum_{j \in S} w'_j = v,}$$

with the same feasibility notion and the same recursion structure as (2), transposed to track minimum time for a target scaled value rather than maximum value for a target time; the

recursion is

$$g(i, v) = \min\left(g(i-1, v), \lfloor g(i-1, v - w'_i) + p_i \leq d_i \rfloor \cdot (g(i-1, v - w'_i) + p_i)\right),$$

by the same argument as in Theorem 3's proof, with the roles of the time and value axes exchanged. This table has $O(n \cdot n^2/\epsilon) = O(n^3/\epsilon)$ entries, each computed in $O(1)$ time, so the whole computation runs in time polynomial in n and $1/\epsilon$.

Let $\hat{S}$ be the feasible subset of largest scaled value v with $g(n, v) < \infty$, and let S^* be the true optimal subset (value $W^* = \sum_{i \in S^*} w_i$). We consider two cases, according to which argument of $K = \max(1, \epsilon w_{\max}/n)$ attains the maximum.

Case $\epsilon w_{\max}/n < 1$. Then $K = 1$, so $w'_i = \lfloor w_i \rfloor = w_i$ for every i (the w_i are integers by Definition 1): the scaling is vacuous, the dynamic program above is exactly Theorem 3's dynamic program, and it returns $\hat{S} = S^*$, so $W(\hat{S}) = W^* \geq (1 - \epsilon)W^*$ trivially (as $\epsilon \in (0, 1)$ and $W^* \geq 0$).

Case $\epsilon w_{\max}/n \geq 1$. Then $K = \epsilon w_{\max}/n$ exactly (not merely an upper bound), so $Kn = \epsilon w_{\max}$ exactly. Since $w'_i \leq w_i/K$, we have $\sum_{i \in S^*} w'_i \geq \sum_{i \in S^*} (w_i/K - 1) \geq W^*/K - n$, and S^* is itself a feasible subset achieving this scaled value or better, so the returned $\hat{S}$ satisfies $\sum_{i \in \hat{S}} w'_i \geq W^*/K - n$. Converting back, since $w_i \leq K(w'_i + 1)$,

$$\begin{aligned} W(\hat{S}) &= \sum_{i \in \hat{S}} w_i \geq K \sum_{i \in \hat{S}} w'_i \geq K \left(\frac{W^*}{K} - n \right) = W^* - Kn \\ &= W^* - \epsilon w_{\max} \geq W^* - \epsilon W^* = (1 - \epsilon)W^*, \end{aligned}$$

using $Kn = \epsilon w_{\max}$ (this case) and $w_{\max} \leq W^*$ (Assumption 1). Either case gives $W(\hat{S}) \geq (1 - \epsilon)W^*$. $\square$

Example 4 (The FPTAS on the running instance) Apply Theorem 4 to Example 1 with a deliberately loose $\epsilon = 0.5$. The scaling factor is $K = \max(1, 0.5 \cdot 10/4) = 1.25$, and the scaled weights are $w' = (\lfloor 5/1.25 \rfloor, \lfloor 8/1.25 \rfloor, \lfloor 3/1.25 \rfloor, \lfloor 10/1.25 \rfloor) = (4, 6, 2, 8)$. Re-running the dynamic program on these scaled weights (tracking minimum dispatch time per scaled value, as in the proof) again selects $S = \{1, 2, 4\}$, recovering the true optimum $W^* = 23$ exactly—comfortably inside the guaranteed bound $(1 - \epsilon)W^* = 11.5$. This mirrors Section 5's finding at scale: the FPTAS's worst-case bound is rarely tight in practice, even at a loose ϵ . Here the scaling is active ($K > 1$) yet harmless; Section 5.3 shows that this is typical of random instances, while Proposition 8 gives instances on which the guarantee is nearly attained.

Algorithm 2 states the FPTAS's value-indexed construction explicitly. It has exactly the same shape as Algorithm 1, with the roles of the time axis and the (scaled) value axis exchanged, which is why its table size no longer depends on P .

Remark 3 (Zero-scaled sites and an optional completion step) When $K > 1$ a site with $w_i < K$ has scaled weight $w'_i = 0$ and, because Algorithm 2 minimizes dispatch time for each scaled value, such a site is never selected. The bound of Theorem 4 accounts for the weight so lost (at most K per site), and Proposition 8 shows the loss can approach ϵ of the optimum. A natural post-processing step, which cannot decrease the returned weight and so preserves Theorem 4, is to re-insert rejected sites in decreasing order of weight while the set stays feasible (checkable in $O(n)$ by Lemma 1); we use it as the “completed” variant in Section 5.3.

Algorithm 2 FPTAS for MWHED (Theorem 4)

Require: sites $1, \dots, n$ with $d_1 \leq \dots \leq d_n$, with dispatch times p_i , deadlines d_i , weights w_i , accuracy $\epsilon \in (0, 1)$

Ensure: a dispatch order with on-time weight $\geq (1 - \epsilon)W^*$

```
1: delete every site with  $p_i > d_i$ ; relabel so  $d_1 \leq \dots \leq d_n$ 
2:  $w_{\max} \leftarrow \max_i w_i$ ;  $K \leftarrow \max(1, \epsilon w_{\max}/n)$ 
3:  $w'_i \leftarrow \lfloor w_i/K \rfloor$  for  $i = 1, \dots, n$ ;  $V \leftarrow \sum_{i=1}^n w'_i$ 
4:  $g[0][0] \leftarrow 0$ ;  $g[0][v] \leftarrow +\infty$  for  $v = 1, \dots, V$ 
5: for  $i = 1, \dots, n$  do
6:   for  $v = 0, \dots, V$  do
7:      $g[i][v] \leftarrow g[i-1][v]$   $\triangleright$  exclude site  $i$ 
8:     if  $v \geq w'_i$  and  $g[i-1][v-w'_i] < +\infty$  and  $g[i-1][v-w'_i] + p_i \leq d_i$  then
9:       if  $g[i-1][v-w'_i] + p_i < g[i][v]$  then
10:         $g[i][v] \leftarrow g[i-1][v-w'_i] + p_i$ 
11:         $choice[i][v] \leftarrow \text{TRUE}$ 
12:       end if
13:     end if
14:   end for
15: end for
16:  $v^* \leftarrow \max\{v : g[n][v] < +\infty\}$ 
17:  $\hat{S} \leftarrow \emptyset$ ;  $v \leftarrow v^*$ 
18: for  $i = n, \dots, 1$  do  $\triangleright$  backtrack, as in Algorithm 1
19:   if  $choice[i][v]$  then
20:      $\hat{S} \leftarrow \hat{S} \cup \{i\}$ ;  $v \leftarrow v - w'_i$ 
21:   end if
22: end for
23: return  $\hat{S}$ , dispatched in increasing-deadline order
```

4.4 A second tractable special case: equal dispatch times

Theorem 2 shows that heterogeneous dispatch costs, weighed against heterogeneous criticality under a shared deadline, already suffice for hardness. The next result shows that the reverse axis—heterogeneous deadlines under a shared dispatch cost—does not. We state it for $m \geq 1$ identical vehicles. In *MWHED- m* each of m vehicles, all starting at the depot at time 0, serves a sequence of round trips; each site is served by at most one vehicle, takes the occupancy p_i regardless of the vehicle, and is on time if its completion time on its vehicle is at most d_i . MWHED-1 is MWHED.

For readers who do not work with matroids we recall the notions used. A *matroid* on a finite ground set E is a family $\mathcal{I}$ of *independent* subsets of E with (i) $\emptyset \in \mathcal{I}$; (ii) if $B \in \mathcal{I}$ and $A \subseteq B$ then $A \in \mathcal{I}$; and (iii) the *augmentation property*: if $A, B \in \mathcal{I}$ and $|A| < |B|$, then $A \cup \{x\} \in \mathcal{I}$ for some $x \in B \setminus A$. The *greedy algorithm* considers the elements of E in order of non-increasing weight and keeps an element if the set kept so far plus that element is still independent. The *transversal matroid* of a bipartite graph has as independent sets the subsets of one side that can be matched into the other side; it is used here with sites on one side and (vehicle, position)

slots on the other. We prove the matroid property we need directly rather than cite it. A *union-find* (disjoint-set) structure maintains a partition of $\{0, 1, \dots, n\}$ under FIND, which returns the representative of a block, and UNION, which merges two blocks; with path compression a sequence of n operations costs $O(n\alpha(n))$ time (Korte and Vygen 2018), where α is the inverse Ackermann function, which grows more slowly than any iterated logarithm and is at most 4 for every input size that can occur in practice.

Theorem 5 (Equal dispatch cost is tractable, for any number of identical vehicles) *If $p_i = p$ for all $p_i = p$ for all i (all sites equidistant from the depot, in round-trip dispatch time), MWHED-MWHED- m is solvable exactly in $O(n \log n)$ time $O(n \log n)$ time for every $m \geq 1$. In particular this holds for MWHED ($m = 1$).*

Proof ~~With equal dispatch costs, the k~~ *Step 1: positions and slots.* Delete sites with $p > d_i$ (Assumption 1) and put $D_i := \min(\lfloor d_i/p \rfloor, n) \geq 1$. On any vehicle the site served j -th dispatched site (whichever it is) completes at time kp , regardless of order. Assigning a site to “dispatch position” k is therefore feasible for that site if and only if $kp \leq d_i$, i.e., $k \leq \lfloor d_i/p \rfloor =: D_i$. The problem becomes: choose an injective assignment of a subset of sites to positions $1, \dots, n$, each site i assigned to a position at most D_i , maximizing the total weight of assigned sites $\sum p$, so it is on time iff $j \leq D_i$. A set S of sites is *feasible* if all its sites can be served on time. Call (v, j) the slot of vehicle v at position j . Then S is feasible iff its sites can be assigned to distinct slots with each site i at a position $\leq D_i$: given a schedule, assign each site to the slot it occupies; conversely, given such an assignment, serve on each vehicle its assigned sites in position order, shifting them to consecutive positions $1, 2, \dots$, which only lowers positions.

Step 2: a counting criterion. For a set S and $k = 1, \dots, n$ let $N_S(k) := |\{i \in S : D_i \leq k\}|$. We claim S is feasible iff $N_S(k) \leq mk$ for all k . ($\Rightarrow$) The $N_S(k)$ sites with $D_i \leq k$ need distinct slots at positions $\leq k$, and there are mk such slots. ($\Leftarrow$) Order S by non-decreasing D as $s_1, \dots, s_t$ and give s_r the slot at position $\lceil r/m \rceil$ (filling m vehicles at each position in turn). It is on time iff $\lceil r/m \rceil \leq D_{s_r}$; and $r \leq N_S(D_{s_r}) \leq mD_{s_r}$, so $r/m \leq D_{s_r}$ and, D_{s_r} being an integer, $\lceil r/m \rceil \leq D_{s_r}$. This is the classical problem of scheduling unit-time jobs with deadlines to maximize total profit. The family of subsets that admit such a feasible assignment forms a matroid (the transversal matroid of the bipartite feasibility relation between sites and positions).

Step 3: the feasible sets form a matroid. Properties (i) and (ii) are clear from the criterion. For (iii) let A, B be feasible with $|A| < |B|$. Let k be the largest integer in $\{0, \dots, n\}$ with $N_B(k) \leq N_A(k)$ (with $N(0) := 0$, so k exists); as $N_B(n) = |B| > |A| = N_A(n)$ we have $k < n$, and by maximality $N_B(j) \geq N_A(j) + 1$ for all $j > k$. Subtracting $N_B(k) \leq N_A(k)$ from $N_B(k+1) \geq N_A(k+1) + 1$ shows that B has more sites with $D = k+1$ than A has, so some $x \in B \setminus A$ has $D_x = k+1$. Adding x to A leaves $N(j)$ unchanged for $j \leq k$ and raises it by one for $j \geq k+1$, where $N_A(j) + 1 \leq N_B(j) \leq mj$ since B is feasible. By Step 2, $A \cup \{x\}$ is feasible.

Step 4: greedy is optimal. Let $G = \{g_1, \dots, g_r\}$ be the set the greedy algorithm returns (weights $w_{g_1} \geq \dots \geq w_{g_r}$, ties broken by index), and let $O = \{o_1, \dots, o_s\}$ be any feasible set with $w_{o_1} \geq \dots \geq w_{o_s}$. G is maximal feasible, so augmentation gives $s \leq r$. We claim $w_{g_k} \geq w_{o_k}$ for all $k \leq s$, which proves $w(G) \geq w(O)$. Suppose $w_{g_k} < w_{o_k}$ for some k . Apply augmentation to $A = \{g_1, \dots, g_{k-1}\}$ and on any matroid, the greedy algorithm—here: sort

$B = \{o_1, \dots, o_k\}$: some $o_j \in B \setminus A$ makes $A \cup \{o_j\}$ feasible, and $w_{o_j} > w_{o_k} > w_{g_k}$. The greedy algorithm examined o_j before g_k , when its kept set was a subset of A ; by property (ii) that set plus o_j was feasible, so greedy kept o_j , i.e. $o_j \in G$ and, being examined before g_k , $o_j \in A$ —a contradiction.

Step 5: implementation. Sort the sites by decreasing w_i , and assign each in turn to the latest available position $k \leq D_i$, or leave it unassigned if none is available—returns a maximum-weight independent set, i.e., an optimal solution (Korte and Vygen 2018, Ch. 13). Sorting takes $O(n \log n)$ time, and maintaining the set of available positions under repeated “find and remove the latest available position $\leq D_i$ ” queries can be implemented with a weight, $O(n \log n)$. Maintain for each position $q \in \{1, \dots, n\}$ its number of free slots, initially m , and a union-find structure in $O(n\alpha(n))$ additional time, giving $O(n \log n)$ overall structure over $\{0, \dots, n\}$ in which $\text{FIND}(q)$ is the largest position $\leq q$ with a free slot (or 0 if none): when position q has no free slot left, merge it into $q - 1$. To process site i , let $q := \text{FIND}(D_i)$; if $q \geq 1$ keep i and use one slot of q , otherwise discard i . This is Algorithm 3. It keeps i exactly when the kept set plus i is feasible. If $q \geq 1$ the construction itself exhibits a feasible assignment. If $q = 0$, let q^* be the largest $q > D_i$ such that all slots at positions $1, \dots, q$ are full. Every kept site occupying a slot at a position $\leq q^*$ has $D < q^*$: a kept site with $D > q^*$ would have been given the latest free slot at a position $\leq D$, and position $q^* + 1$ was free then (it is free now), so it would sit above q^* . Hence the kept set has $N(q^*) \geq mq^*$ and, since $D_i \leq q^*$, adding i violates Step 2’s criterion. The union-find operations cost $O(n\alpha(n))$, so the total is $O(n \log n)$. $\square$

Example 5 (The matroid greedy on an equal-cost instance) Consider four sites all at the same round-trip distance $p = 2$ from the depot, with deadlines $d = (3, 3, 5, 9)$ and weights $w = (9, 4, 7, 6)$, so the latest feasible dispatch position for each site is $D_i = \lfloor d_i/p \rfloor = (1, 1, 2, 4)$. ~~Sorted~~ With one vehicle ($m = 1$), sorted by decreasing weight, the greedy order is site 1 ($w = 9$), site 3 ($w = 7$), site 4 ($w = 6$), site 2 ($w = 4$). Site 1 takes position 1 (~~its only option~~); site 3 takes position 2; site 4 takes position 4; site 2 needs a position ≤ 1 , but position 1 is already taken, so ~~site 2 it~~ is left unassigned. The greedy solution dispatches sites 1, 3, 4 for a total weight of $9 + 7 + 6 = 22$; ~~sacrificing only the lowest-weight site whose deadline conflicts with a higher-weight site’s—exhaustive;~~ exhaustive search over all 16 subsets confirms 22 is ~~indeed optimal—optimal~~. With two vehicles ($m = 2$) position 1 has two slots, so site 2 also fits: all four sites are served, for 26 (vehicle A serves site 1 then site 3, completing at 2 and 4; vehicle B serves site 2 then site 4, completing at 2 and 4).

Remark 4 Theorems 2 and 5 ~~together~~ show that neither heterogeneous dispatch cost nor heterogeneous deadlines alone is responsible for MWHED’s hardness: ~~it is specifically the interaction between heterogeneous dispatch cost and heterogeneous criticality, competing under a shared deadline structure, that produces a knapsack-hard problem; Theorem 9 below makes this exact.~~ A dispatch network in which every site is equally far from the depot—a plausible approximation for, e.g., a ring of substations around a central facility, or short-radius urban service territories—remains exactly solvable regardless of how heterogeneous the sites’ hazard deadlines or criticalities are, and regardless of the number of identical vehicles.

Algorithm 3 Matroid greedy for equal-cost MWHED- m (Theorem 5)

Require: sites $1, \dots, n$ with deadlines d_i and weights w_i , common dispatch time p ,
deadlines d_i , weights w_i , number of identical vehicles m

Ensure: an optimal on-time subset S^*

```
1:  $D_i \leftarrow \lfloor d_i/p \rfloor$  for  $i = 1, \dots, n$  sort sites into order  $\pi_1, \dots, \pi_n$  with  
    $w_{\pi_1} \geq w_{\pi_2} \geq \dots \geq w_{\pi_n}$  delete sites with  $p > d_i$ ;  $D_i \leftarrow \min(\lfloor d_i/p \rfloor, n)$   $\triangleright$  latest  
   feasible position  
2:  $available \leftarrow \{1, \dots, n\}$   $free[q] \leftarrow m$  for  $q = 1, \dots, n$ ;  $S^* \leftarrow \emptyset$   $i \leftarrow \pi_k$  FIND over  
    $\{0, \dots, n\}$ : every  $q$  its own block;  $S^* \leftarrow \emptyset$   
3: for each site  $i$  in order of non-increasing  $w_i$  do  
4:    $q^* \leftarrow \max\{q \in available : q \leq D_i\}$   $q \leftarrow \text{FIND}(D_i)$   $\triangleright$  largest position  $\leq D_i$  with  
   a free slot, or 0  
5:   if  $q \geq 1$  then  
6:      $S^* \leftarrow S^* \cup \{i\}$   $S^* \leftarrow S^* \cup \{i\}$ ;  $available \leftarrow available \setminus \{q^*\}$   
      $free[q] \leftarrow free[q] - 1$   
7:     if  $free[q] = 0$  then  
8:        $\text{UNION}(q, q-1)$   $\triangleright q$  is now represented by  $q-1$   
9:     end if  
10:  end if  
11: end for  
12: return  $S^*$ , dispatched in increasing-deadline order (on one vehicle) or position  
    order (on  $m$ )
```

4.5 The cost of not reconsidering

Theorems 3 and 4 give planners exact and near-exact tools, but a planner who instead falls back on the simplest conceivable heuristic—dispatch might instead fall back on a simple rule. The simplest is to dispatch sites in increasing-deadline order and simply accept whichever happen to still be on time, never skipping a site to make room for a later, more valuable one—receives no guarantee at all (naive EDD: a site that is already late still consumes its dispatch time). A slightly smarter rule skips a site that would arrive late (EDD with skipping). Neither has any guarantee.

Proposition 6 (Deadline-order dispatch has unbounded worst-case ratio) *For every $r \in (0, 1)$ there is an MWHED instance, satisfying Assumption 1, on which the naive policy's both naive EDD and EDD with skipping obtain on-time weight $< r \cdot W^*$.*

Proof Fix $r \in (0, 1)$ and choose an integer $W > 1/r$. Consider the two-site instance with $p = (1, 2)$, $d = (1, 2)$, $w = (1, W)$; both sites satisfy Assumption 1 ($p_i = d_i$). Since $d_1 < d_2$, the naive policy dispatches both rules dispatch site 1 first, completing at time $1 \leq d_1$ (on time, weight 1); then site 2, completing would then complete at time $1 + 2 = 3 > d_2 = 2$ (exposed, losing weight W); its: naive EDD serves it anyway and it is exposed, and EDD with skipping discards it; either way the total is 1. Dispatching site 2 alone instead completes it at time $2 \leq d_2$, on time, for weight W ; this is feasible and, since $W > 1 \geq$ the value of any

subset not containing site 2, it since $W > 1$ this is optimal, so $W^* = W$. The naive policy's ratio is therefore $1/W < r$. $\square$

Proposition 6 is not merely a boundary case: the two sites here differ only by a factor of two in dispatch time, yet the naive policy's loss can be made arbitrarily large by scaling W alone. No fixed constant-factor guarantee is possible for a policy that never reconsiders an early, low-value commitment against a later, high-value one—exactly the reconsideration that Theorem 3's dynamic program and one; skipping doomed sites removes only the waste of serving a site that is already late. It is natural to ask whether a local repair rule avoids the problem. The weighted generalization of Moore–Hodgson's rule used as a heuristic in Section 5—process sites in deadline order and, whenever the running schedule becomes infeasible, discard the scheduled site of smallest weight-to-time ratio—does very well on random instances, but it too has no guarantee.

Algorithm 4 Weighted greedy repair (heuristic; no guarantee)

Require: sites with $d_1 \leq \dots \leq d_n$ (after deleting sites with $p_i > d_i$), dispatch times p_i , weights w_i

Ensure: a feasible on-time subset *kept* (not necessarily optimal)

```

1: kept  $\leftarrow \emptyset$ ; total  $\leftarrow 0$ 
2: for  $i = 1, \dots, n$  do
3:   kept  $\leftarrow \text{kept} \cup \{i\}$ ; total  $\leftarrow \text{total} + p_i$ 
4:   while total  $> d_i$  and kept  $\neq \emptyset$  do
5:      $j \leftarrow \arg \min_{j' \in \text{kept}} w_{j'} / p_{j'}$   $\triangleright$  smallest weight-to-time ratio
6:     kept  $\leftarrow \text{kept} \setminus \{j\}$ ; total  $\leftarrow \text{total} - p_j$ 
7:   end while
8: end for
9: return kept, dispatched in increasing-deadline order

```

Its output is always feasible, by the same inductive argument as for the unweighted algorithm: removing a site can only shorten the completion time of every site that remains.

Proposition 7 (Weighted greedy repair has unbounded worst-case ratio) *For every integer $k \geq 2$ the two-site instance $p = (1, k)$, $d = (1, k)$, $w = (2, 2k - 1)$ satisfies Assumption 1 and the weighted greedy repair of Algorithm 4 returns on-time weight 2, while $W^* = 2k - 1$; the ratio $2/(2k - 1) \rightarrow 0$.*

Proof The algorithm keeps site 1 (total $1 \leq d_1$). On adding site 2 the total becomes $1 + k > d_2 = k$, so it discards the kept site of smallest ratio w_j/p_j : site 1 has ratio 2 and site 2 has ratio $(2k - 1)/k < 2$, so site 2 is discarded and the result is $\{1\}$ with weight 2. Site 2

alone completes at $k \leq d_2$, so $W^* > 2k - 1$, and the two sites cannot both be on time, so $W^* = 2k - 1$ for $k \geq 2$. $\square$

These two propositions say that some form of reconsideration governed by the weights *and* the dispatch times is needed for any worst-case guarantee. They do not say that the dynamic program or the FPTAS is the only way to obtain one: other polynomial algorithms with guarantees may exist, and we do not claim otherwise.

4.6 Further structural results

Tightness of the FPTAS analysis.

The proof of Theorem 4 ~~'s FPTAS perform,~~ loses at most $Kn = \epsilon w_{\max}$ and then bounds this by ϵW^* . The next result shows this is nearly the truth for Algorithm 2 as stated.

Proposition 8 (Tightness of the value-scaling analysis) *Let $\epsilon \in (0, 1)$, let $n \geq 2$ and let $M \geq 2n/\epsilon$ be an integer. There is an instance with n sites, satisfying Assumption 1, on which Algorithm 2 returns weight M while $W^* = M + (n-1)(\lceil K \rceil - 1)$ with $K = \epsilon M/n$. Hence its ratio is at most $(1 + \epsilon \frac{n-1}{n} - \frac{n-1}{M})^{-1}$ which tends to $1/(1+\epsilon)$ as $n, M \rightarrow \infty$. Theorem 4 guarantees $1 - \epsilon$, and $\frac{1/(1+\epsilon)}{1-\epsilon} = \frac{1}{1-\epsilon^2}$, so the guarantee is tight up to a factor $1 - \epsilon^2$.*

Proof Take $p_i = 1$ and ~~that Example 1 already previewed at a modest scale. Section 5 confirms the same qualitative gap persists, if less extremely, on random instances of realistic size.~~ $d_i = n$ for all i , $w_1 = M$, and $w_i = \lceil K \rceil - 1$ (> 1) for $i \geq 2$. All n sites can be served (total time n , every deadline n), so $W^* = M + (n-1)(\lceil K \rceil - 1)$. The scaling factor is $K = \epsilon M/n > 2 > 1$, site 1 has scaled weight $\lfloor n/\epsilon \rfloor$, and every other site has $w_i < K$, hence scaled weight 0. Algorithm 2 minimizes the dispatch time for each scaled value, so it never selects a zero-scaled site (adding it only increases the time at an unchanged value) and returns $\{1\}$, of weight M . Finally $\lceil K \rceil - 1 > K - 1$ gives the stated bound. $\square$

The optional completion step of Remark 3 repairs this family completely, which is why we report both variants in Section 5.3.

~~We illustrate Theorems 3 and 4 and Proposition 6 in five complementary experiments: an accuracy comparison across four algorithms on synthetic instances; a runtime comparison isolating the pseudo-polynomial (~~

Which heterogeneity causes hardness?

Theorems 2 and 5 and Moore's algorithm together settle the complexity of every restriction obtained by making some of the three data vectors constant. For $T \subseteq \{p, w, d\}$ let $\mathcal{C}_T$ be the class of MWHED instances in which each vector named in T is constant ($p_i = p$, $w_i = w$, $d_i = D$ for all i).

Theorem 9 (Classification by homogeneity) *Let $T \subseteq \{p, w, d\}$.*

(a) If $p \in T$ or $w \in T$, MWHED restricted to $\mathcal{C}_T$ is solvable in $O(n \log n)$ time.

(b) If $T \subseteq \{d\}$, MWHEd restricted to $\mathcal{C}_T$ is weakly NP-hard; in particular it is NP-hard for $T = \{d\}$.

Thus an MWHEd restriction is polynomial exactly when the dispatch times or the weights are constant, and constancy of the deadlines never helps.

Proof (a) If $p \in T$, apply Theorem 5 with $m = 1$. If $w \in T$, all sites have the same weight w , so maximizing on-time weight means maximizing the number of on-time sites, which is the problem $1\|\sum U_j$ solved in $O(n \log n)$ time by Moore's algorithm (Moore 1968). (b) $\mathcal{C}_{\{d\}} \supseteq$ the instances of Theorem 2, so it is NP-hard, and it is weakly NP-hard because Theorem 3 ~~) versus truly polynomial (Theorem 4) dependence on the instance's numerical magnitude; a sensitivity sweep over~~ solves it in pseudo-polynomial time; $\mathcal{C}_\emptyset$ contains $\mathcal{C}_{\{d\}}$. $\square$

The classification is elementary once Theorems 2 and 5 are available; its value is interpretive. Hardness needs *two independent* sources of heterogeneity, in the dispatch time *and* in the criticality, which compete for the shared time budget; heterogeneity in the hazard deadlines, the part of the instance produced by the hazard model, plays no role. In network terms: if every site is equally far from the depot, or every site is equally critical, the dispatch problem is easy however the hazard spreads.

5 Numerical study

The study has four purposes: to check every algorithm against an independent exhaustive search (Section 5.1); to compare the algorithms and the heuristics of Section 4 across instance sizes and generators, and to show, in a regime where the FPTAS's ~~accuracy parameter rounding~~ is genuinely active, how its accuracy and cost behave as ϵ ; ~~a robustness check repeating the accuracy comparison under two alternative synthetic instance-generation regimes; and, finally, a small instance built directly from public records of~~ shrinks (Sections 5.2–5.5); and to illustrate the model on a real wildfire, with a sensitivity analysis over its uncertain inputs (Section 5.6), ~~to confirm the synthetic findings are not an artifact of how the synthetic instances are generated.~~ All code is self-contained and deterministic given its seeds (see the Data Availability statement); every table below is regenerated by `python3 experiment.py` (about 15 seconds) and `python3 case_study_campfire.py`.

5.1 Verification against exhaustive search

Before any performance claim we verify the implementations (`verify_small.py`). (i) On 400 random instances with $n \leq 7$ the dynamic program's optimum equals the optimum over *all* $n!$ dispatch orders, which checks Lemma 1 and Theorem 3 from the definition. (ii) On 1,500 random instances with $n \leq 14$ (a third of them with weights up to 10^6 and all with individually infeasible sites left in) the dynamic program equals the best feasible subset found by enumeration, and the FPTAS returns a feasible set of weight at least $(1 - \epsilon)W^*$ for $\epsilon \in \{0.5, 0.2, 0.05\}$, with and without the completion step, with no violation in 4,500 runs (2,923 of which had active scaling). (iii) The equal-cost greedy of Algorithm 3 equals the dynamic program on 300 instances for $m = 1$, and

equals exhaustive search over all assignments to $m = 2, 3$ vehicles on the instances with $n \leq 7$. (iv) Proposition 7's ratio $2/(2k - 1)$ is reproduced for $k = 3, 10, 100$.

5.2 Accuracy across instance sizes

For $n \in \{10, 15, 20, 25, 30, 50, 75, 100\}$ we generate 20 random instances each (p_i uniform on $\{1, \dots, 20\}$, w_i uniform on $\{1, \dots, 100\}$, d_i uniform on $\{1, \dots, \sum_i p_i\}$); and compare four algorithms, individually infeasible sites deleted as in Assumption 1) and compare: the true optimum via (Theorem 3's exact dynamic program); the FPTAS of Theorem 4 at $\epsilon \in \{0.1, 0.2\}$, implemented exactly at $\epsilon \in \{0.1, 0.2\}$, implemented as in the proof (the value-indexed dual DP, not a shortcut); a weighted Moore-Hodgson-style greedy repair heuristic (dispatch in earliest-deadline order, and whenever the running schedule becomes infeasible, repeatedly drop the scheduled site with the smallest weight-to-time ratio until feasible again—a natural, always-feasible heuristic generalizing Moore's unweighted rule, for which we claim no approximation guarantee; the weighted greedy repair (Algorithm 4); and the naive baseline of two deadline-order baselines, naive EDD and EDD with skipping (Proposition 6 (dispatch in earliest-deadline order, simply keep whichever sites happen to still be on time, with no repair at all)). Naive EDD keeps dispatching to sites that are already late, so it is a weak anchor; EDD with skipping is the fairer simple baseline. Table 4 reports the ratio of each method's on-time weight to the true optimum, averaged over the 20 instances at each size. Algorithm 4 states the greedy repair heuristic precisely; feasibility of its output is guaranteed by the same inductive argument used for the unweighted Moore-Hodgson algorithm (each repair step can only shorten, never lengthen, the completion time of every remaining scheduled site), but—unlike Algorithms 1–3—we prove no approximation ratio for it; it is included as a realistic, easy-to-implement point of comparison, not as a fifth theorem mean ratios to the optimum, and Figure 2 plots them.

sites $1, \dots, n$ with $d_1 \leq \dots \leq d_n$, dispatch times p_i , weights w_i a feasible on-time subset $kept$ (not necessarily optimal) $kept \leftarrow \emptyset$; $total \leftarrow 0$ $kept \leftarrow kept \cup \{i\}$; $total \leftarrow total + p_i$ $j \leftarrow \arg \min_{j' \in kept} w_{j'}/p_{j'}$ $kept \leftarrow kept \setminus \{j\}$; $total \leftarrow total - p_j$ $kept$, dispatched in increasing-deadline order—

At both approximation levels tested, the The FPTAS recovers the true optimum on essentially every one of the 160 random instances (mean ratio rounds to 1.000 throughout), far outperforming its optimum on every instance, but this is *not* evidence about the approximation scheme. With weights at most 100 the scaling factor $K = \max(1, \epsilon w_{\max}/n)$ equals 1 for $\epsilon = 0.1$ at every $n \geq 15$ and for $\epsilon = 0.2$ at every $n \geq 25$ (last row of Table 4), so no rounding takes place and the FPTAS coincides with the exact value-indexed dynamic program; a ratio of 1.000 is then guaranteed, not informative. The next subsection therefore uses weights large enough that the rounding is active. Among the heuristics, the greedy repair captures 99.3–100% of the optimum on average but has a worst ratio of 0.93 at $n = 10$, consistent with its having no guarantee (Proposition 7); naive EDD shows no trend with n , as a rule that never reconsiders would not, though a worst-case $(1 - \epsilon)$ guarantee, consistent

Table 4 Mean ratio to the true optimum, by instance size n , over 50 random instances per column (weights ≤ 100). The last worst ratio over the 50 instances for greedy repair and for naive EDD, and the share of instances on which the FPTAS’s scaling rounding at all), for $\epsilon = 0.2$ and $\epsilon = 0.1$.

n	10	15 -20	25-30 -50	75 -100	<u>200</u>
FPTAS ($\epsilon = 0.2$)	1.000	1.000	1.000	1.000	1.000
FPTAS ($\epsilon = 0.1$)	1.000	1.000	1.000	1.000	1.000
Greedy repair (M-H style)	0.993	0.997 -0.996	0.993 - 0.997 - 0.998 - 0.999	0.999	0.999
<u>EDD with skipping</u>	<u>0.933</u>	<u>0.941</u>	<u>0.954</u>	<u>0.972</u>	<u>0.981</u>
Naive EDD (no repair)	0.380 - 0.714	0.343 - 0.653	0.622 - 0.595	0.369 - 0.573	0.323 - 0.66
<u>worst ratio, greedy repair</u>	<u>0.515</u> - <u>0.929</u>	<u>0.439</u> - <u>0.963</u>	<u>0.976</u>	<u>0.992</u>	<u>0.995</u>
<u>worst ratio, naive EDD</u>	<u>0.065</u>	<u>0.132</u>	<u>0.032</u>	<u>0.110</u>	<u>0.027</u>
<u>share with $K = 1, \epsilon = 0.2 / 0.1$</u>	<u>0/.54</u>	<u>.60/1</u>	<u>1/1</u>	<u>1/1</u>	<u>1/1</u>

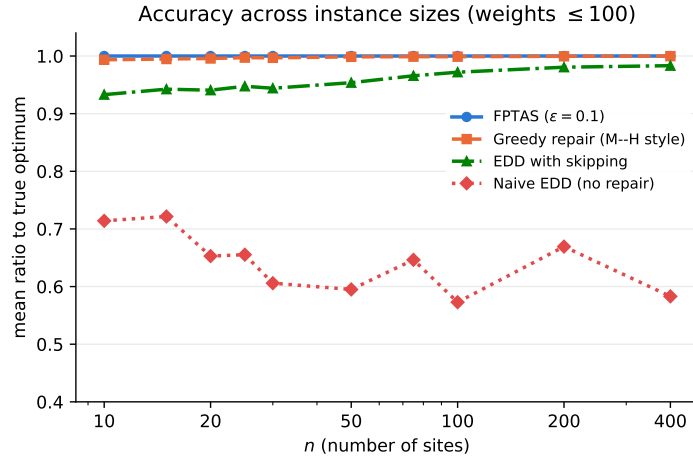


Fig. 2 Table 4 plotted. The FPTAS and greedy repair sit at 1.0; EDD with skipping improves slowly with n ; naive EDD fluctuates around 0.6 with no trend and, on individual instances, captures only a few percent of the optimum.

with the well-documented empirical looseness of knapsack-style FPTAS result such as Proposition 6 does not by itself predict average behavior on random instances.

5.3 The FPTAS with active scaling

We now take weights up to 10^6 with $p_i \leq 20$, so that the time-indexed exact dynamic program stays cheap (its table depends on P , not on the weights) and serves as the reference, while the FPTAS’s scale factor K is in the hundreds to thousands. Three families are used, with $n = 100$ and 30 instances per ϵ for the first two. *Random*: weights uniform on $\{1, \dots, 10^6\}$. *Strongly correlated*: a common deadline and $w_i = 1000p_i + 500$ with p_i up to 1000, the hard family for knapsack-type schemes

(Hejl et al. 2022 study this regime for exact algorithms). *Adversarial*: the family of Proposition 8, with one dominant site and $n - 1$ sites just below the scaling granule. For each, Table 5 reports the worst ratio over the instances for Algorithm 2 as stated and, for the adversarial family, with the completion step of Remark 3.

Table 5 FPTAS accuracy with active scaling ($n = 100$, $p_i \leq 20$ or as stated). Entries are the *worst* ratio to the optimum over the instances of each family (30 per ϵ for the first two). “Share < 1 ” is the fraction of random instances on which the FPTAS was not exactly optimal. Every entry respects the guarantee $1 - \epsilon$.

ϵ	$1 - \epsilon$	Random	Share < 1	Str. corr.	Adversarial	Adv. + completion
0.50	0.50	0.9998	0.30	0.9993	0.669	1.000
0.30	0.70	0.99996	0.10	0.9994	0.771	1.000
0.20	0.80	0.99996	0.10	0.9994	0.835	1.000
0.10	0.90	0.99999	0.03	0.9996	0.910	1.000
0.05	0.95	1.00000	0.00	0.9996	0.953	1.000

Table 6 Cost of the FPTAS as ϵ shrinks (random weights up to 10^6 , $n = 100$): mean scaling factor K , mean number of dynamic-program cells, and mean wall-clock time. A value-indexed exact table would need about 5×10^7 columns per row at these weights, against the 10^4 – 10^5 used here.

ϵ	0.50	0.30	0.20	0.10	0.05
mean K	4.996	2.997	1.998	999	500
cells ($\times 10^6$)	0.99	1.65	2.47	4.95	9.90
time (ms)	3.4	4.9	6.7	14.4	33.1

Two things are visible. First, on random and on strongly correlated instances the FPTAS is nearly exact even at $\epsilon = 0.5$ (worst ratio 0.9993), far above its guarantee: the bound $\epsilon w_{\max}$ charges every selected site a full rounding error K and compares the total to $w_{\max}$, whereas on these instances the optimum contains many sites and $W^* \gg w_{\max}$. This is why the ratio is close to 1, and it is the answer to the question of why accuracy is near 1 in all earlier tables. Second, the guarantee is *not* vacuous: on the adversarial family, which Proposition 8 shows is the worst case, the ratio falls to 0.669 at $\epsilon = 0.5$ and follows the guarantee as ϵ shrinks, so a planner who needs a worst-case bounds (Ibarra and Kim 1975): the scaling construction is designed to guarantee the worst case, not to describe the typical one. The greedy repair heuristic, despite carrying no proven guarantee, is a substantial practical improvement over doing nothing—capturing 99.3–99.9% of the optimum—because reconsidering even a single locally-worst commitment when infeasibility first arises already recovers most of the value a full optimization would. The naive EDD baseline, which never reconsiders

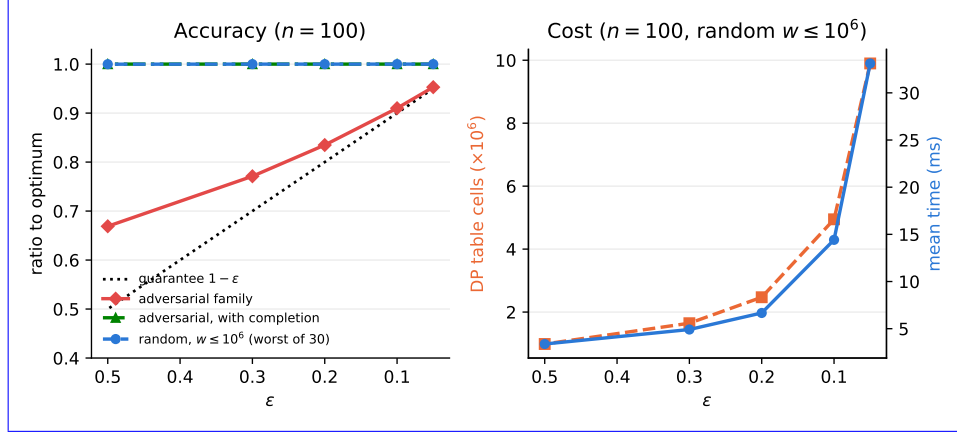


Fig. 3 Tables 5 and 6 plotted. Left: the worst ratio on the adversarial family tracks the guarantee $1 - \epsilon$ (it stays above it and, as Proposition 8 shows, approaches $1/(1 + \epsilon)$); on random instances the FPTAS is essentially exact, and the completion step makes the adversarial family exact. Right: cost grows in proportion to $1/\epsilon$, as the $O(n^3/\epsilon)$ bound says.

anything, captures only 32–62% of the optimal on-time weight and shows no trend of improving with instance size, exactly as Proposition 6 predicts: its worst-case ratio does not shrink with n , it is simply unbounded below on adversarial instances and highly variable on random ones; bound cannot do better than ϵ -dependent cost. The inexpensive completion step removes this failure on the family. Cost grows in proportion to $1/\epsilon$ (Table 6), as predicted, and is independent of P (next subsection).

5.4 Runtime: time-indexed versus value-indexed

Theorem 3’s exact DP dynamic program runs in $O(nP)$ time, where $P = \sum_i p_i$ can be with $P = \sum_i p_i$ exponential in the input’s bit length; Theorem 4’s FPTAS runs in time polynomial in n and $1/\epsilon$, independent of the FPTAS’s table does not depend on P . To see this gap empirically rather than only asymptotically, we fix $n = 40$ and, $\epsilon = 0.1$ and, draw weights up to 10^6 (so the scaling is active at every setting), vary the range $p_{\max}$ from which processing dispatch times are drawn (so P grows roughly linearly with $p_{\max}$), timing, and time both algorithms on 5–5 random instances per setting. Table 7 reports the mean wall-clock time.

Table 7 Mean wall-clock time (milliseconds), $n = 40$, $\epsilon = 0.1$, weights up to 10^6 , over 5 random instances per row, as $p_{\max}$ grows (so P grows roughly proportionally).

$p_{\max}$	50	200	800	3,200	12,800
Exact DP (Theorem 3)	4.7–0.75	16.8–0.93	65.3–2.75	275.7–9.16	1137.9–55.1
FPTAS (Theorem 4)	8.3–1.18	8.4–1.07	7.8–1.17	7.7–1.13	7.6–1.64

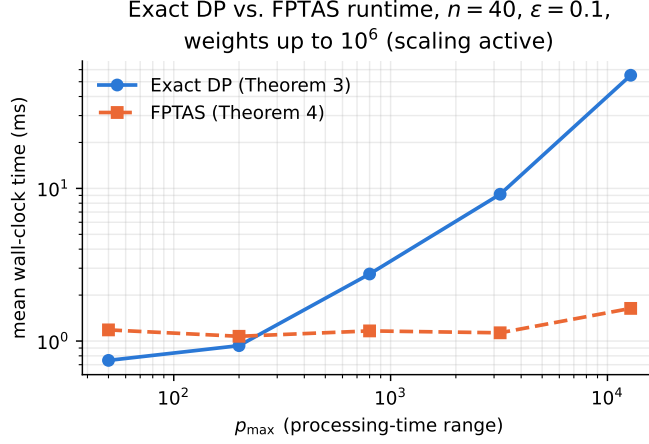


Fig. 4 Table 7 on log-log axes: the exact DP’s time grows with $p_{\max}$ (roughly linearly or slightly faster once the fixed overhead is exceeded), while the FPTAS’s stays flat.

The exact DP’s runtime grows essentially linearly with $p_{\max}$, as its $O(nP)$ bound predicts, increasing roughly 240-fold as $p_{\max}$ increases grows 256-fold; the FPTAS’s runtime stays flat, within measurement noise, across the same range, since its table size $O(n^3/\epsilon)$ never depends on P at all (the smallest settings are dominated by fixed overhead), whereas the FPTAS is flat up to noise. At the smallest $p_{\max}$ tested here the exact DP is actually the exact program is faster in absolute terms, since its (pseudo-polynomial but small) table is cheaper than the FPTAS’s fixed $O(n^3/\epsilon)$ overhead—a reminder that an FPTAS’s asymptotic advantage only pays off once the instance’s approximation scheme pays off only when the numerical magnitude is large enough, exactly the regime Theorem 2’s weak-NP-hardness result says the exact DP cannot handle efficiently.

Theorem 4’s table has $O(n^3/\epsilon)$ entries, so shrinking ϵ should, in principle, grow both the accuracy and the running time of the FPTAS. We fix $n = 30$ and sweep $\epsilon \in \{0.5, 0.3, 0.2, 0.1, 0.05, 0.01\}$ on the same 20 random instances at each value, reporting the mean ratio to the true optimum and the mean wall-clock time in Table ??.

ϵ	0.50	0.30	0.20	0.10	0.05	0.01	Mean ratio to optimum
	0.9999	1.0000	1.0000	1.0000	1.0000	1.0000	
Mean time (ms)	2.5	3.9	3.9	3.8	4.2	3.8	

Even at the loosest setting tested, $\epsilon = 0.5$ (a 50% worst-case loss allowed), the FPTAS already recovers 99.99% of the optimum on average, and by $\epsilon = 0.3$ it matches the optimum on every instance in this sample; consistent with Example 4 and Section 5’s accuracy table, the worst-case bound $(1 - \epsilon)W^*$ is a safety margin that is rarely approached in practice, not a description of typical behavior. The runtime grows only mildly over this range at $n = 30$ (a two-order-of-magnitude shrinkage in ϵ , from 0.5 to 0.01, produces roughly a 50% increase in running time, not the $50\times$ the $O(n^3/\epsilon)$ bound alone would suggest as an upper bound), because the table’s row dimension is capped by $\sum_i w'_i$, which grows sub-linearly in $1/\epsilon$ once the scaled weights $w'_i = \lfloor w_i/K \rfloor$ saturate at small integer values; the $O(n^3/\epsilon)$ bound is a worst-case ceiling; on these

instances the effective table is considerably smaller. We stress that this comparison is between a time-indexed and a value-indexed table, and that the dependence on the weights has simply moved: a value-indexed *exact* program would depend on $\sum_i w_i$ in the same way.

5.5 Robustness to the instance-generation regime

The accuracy comparison of Section 5.1 uses uniformly distributed weights and a deadline horizon spanning the full range $[1, \sum_i p_i]$. To check that its qualitative pattern the pattern of Table 4 is not an artifact of that specific choice, the generator we repeat the comparison at $n = 30$ under two different regimes: (20 instances each) under heavy-tailed weights (drawn from an exponential distribution rather than uniform, so a few sites are far more critical than the rest exponential, mean 20) and tight deadlines (the deadline horizon narrowed to $0.5 \sum_i p_i$, so on average only about half the sites can be served regardless of order). Table 8 reports the same three ratios as before under each regime. $0.5 \sum_i p_i$.

Table 8 Mean ratio to the true optimum under two alternative instance-generation regimes, $n = 30$, 20 random instances per row. “Repair” is the greedy repair of Algorithm 4; “Skip” is EDD with skipping; the FPTAS uses $\epsilon = 0.1$.

Regime	FPTAS ($\epsilon=0.1$)	Repair (M-H)	Skip	Naive EDD
Heavy-tailed weights	1.000	0.998 0.999	0.274 0.944	0.537
Tight deadlines ($0.5 \times$ horizon)	1.000	0.988	0.026 0.799	0.125

The FPTAS and the greedy repair heuristic are essentially unaffected : both continue to recover almost all of the optimum, confirming that Section 5.1’s pattern is not an artifact of uniform weights or a loose deadline horizon. The naive EDD baseline, by contrast, gets *worse* under both regimes—down to 2.6% of optimum under tight deadlines—which is exactly what Proposition 6 predicts: tightening deadlines or increasing weight heterogeneity both widen the gap between the single early commitment naive EDD makes and the value it forecloses, since greedy repair is essentially unaffected and EDD with skipping degrades under tight deadlines, where there is less slack to absorb a poor early choice and more value concentrated in whichever sites get displaced by it : commitment; naive EDD collapses to 12.5% of the optimum, largely because it also wastes time on sites that are already late. (As in Section 5.2, the FPTAS entries are 1.000 partly because the scaling is vacuous at weights ≤ 100 .)

5.6 A real-world illustration: the 2018 Camp Fire

The instances above are synthetic by design—MWHED design: MWHED is a new problem with no existing benchmark set, and the robustness check just given specifically needs controllable distributional properties that only a generator, not a single fixed dataset, can provide experiments need controllable distributions. To ground

the model’s scale ~~and behavior~~ in a real event ~~; we also we~~ build one small instance ~~directly~~ from public records of the 2018 Camp Fire (Butte County, California), the deadliest wildfire in California history (85 fatalities, over 18,000 structures destroyed). We ~~emphasize which numbers below are real measurements and which are separate real measurements from~~ disclosed modeling assumptions, ~~rather than blending the two silently~~ and we present the case as an *illustration of the model and its sensitivity*, not as an operational recommendation.

Real inputs. The fire’s ignition point (a PG&E transmission tower near Pulga, CA) and its progression timeline—in particular, fire reaching Concow approximately 52 minutes after ~~ignition~~ the time zero of the timeline, and spot fires igniting in Paradise approximately 71 minutes after ~~ignition~~ —are as documented in it— are taken from a National Institute of Standards and Technology forensic reconstruction (Maranghides et al. 2021). We take four real, named communities threatened by the fire (Concow, Paradise, Magalia, and Yankee Hill, Butte County) with their real geographic coordinates and 2010 U.S. Census populations (the census immediately preceding the fire, used as a proxy criticality weight w_i ; later counts reflect the ~~fire’s~~ aftermath, not the population at risk). A real regional fire-agency depot (Oroville, CA) anchors the dispatch side.

Disclosed modeling assumptions. ~~Two quantities are not directly measured. First, (i) p_i is computed from twice~~ the real great-circle distance between the depot and ~~each site (a real site i (a lower bound on true road distance) divided by an assumed average response speed; we report results at two illustrative speeds, 50 km/h (conservative, winding mountain roads) and 80 km/h (an optimistic highway-speed upper bound). Second, optimistic). (ii) Magalia and Yankee Hill have no directly documented fire-arrival documented arrival~~ timestamp in the source ~~we use~~, so their ~~deadlines d_i arrival times~~ are extrapolated: ~~we compute the real fire-spread the spread~~ rate implied by Concow’s and Paradise’s real ~~distance from ignition and real documented distance from ignition and real~~ arrival time (12.1 and 14.2 km/h ~~respectively, averaging, average~~ 13.1 km/h) ~~; then apply this averaged rate to Magalia’s and Yankee Hill’s own real is applied to their own straight-line~~ distance from the ignition point. ~~Table 9 reports the resulting instance and Table 10 the outcome of running Algorithms 1–4 unmodified on it (code and full source citations: case_study_campfire.py); this extrapolation is isotropic, whereas the real spread was wind-driven and directional. (iii) The crew can leave at the time zero of the timeline, with no detection or dispatch delay. (iv) The arrival reading of Remark 1 is used: a site is protected if the crew arrives before the fire, so the MWHED deadline is $d_i = d_i^{\text{haz}} + p_i/2$. Table 9 gives the instance.~~

At the conservative Results. At 50 km/h response speed, Concow and Magalia are individually infeasible—even dispatched first and alone, a crew from the Oroville depot cannot reach them before the fire does, a sobering but real reflection of how far this fire’s actual spread outran any single vehicle response. The optimal policy recognizes this immediately and the optimum sends the crew directly straight to Paradise, saving all protecting its weight of 26,218 people at risk there; naive earliest-deadline dispatch, recall, tries Concow and Magalia first ~~because~~ their deadlines are earlier, exhausts the time budget on two sites it can never reach, and arrives ~~; any detour first, even the shortest, uses time Paradise’s deadline cannot absorb. Deadline-order rules go to~~

Table 9 The Camp Fire instance. p_i is the round-trip occupancy from the Oroville depot at the two response speeds; d^{haz} is the fire-arrival minute (documented for Concow and Paradise, extrapolated for the others); the MWHED deadline under the arrival reading is $d_i = d^{\text{haz}} + p_i/2$ (rounded), shown for both speeds; w_i is the 2010 census population.

Site	p_i (50km/h)	p_i (80km/h)	d^{haz}	d_i (50)	d_i (80)	w_i
Concow	63	39	52	84	72	710
Magalia	89	55	58	102	86	11,310
Yankee Hill	54	34	64	91	81	333
Paradise	70	44	71	106	93	26,218

Table 10 On-time weight (population) of each algorithm on the Camp Fire instance, arrival reading; the sites served are in parentheses.

	50 km/h	80 km/h
Exact optimum (Thm. 3)	26,218 <u>26,218</u> (Paradise)	26,218 <u>26,928</u> (Concow, Paradise)
FPTAS, $\epsilon = 0.1$ (Thm. 4)	26,218 <u>26,218</u>	26,218 <u>26,928</u>
Greedy repair (M-H style)	26,218 <u>26,218</u>	26,218 <u>26,218</u> (Paradise)
EDD with skipping	<u>710</u> (Concow)	<u>1,043</u> (Concow, Yankee Hill)
Naive EDD (no repair)	0 <u>710</u> (Concow)	710 <u>1,043</u> (Concow, Yankee Hill)

Concow first because its deadline is earlier, protect only its 710 residents, and arrive at Paradise too late to help anyone—an on-the-nose illustration of Proposition 6’s unbounded worst-case loss, not a constructed adversarial example. At the more optimistic 80. At 80 km/h the shorter trips leave room to serve Concow and then Paradise, for 26,928; the weighted greedy repair misses the Concow detour and returns 26,218, 2.6% less than the optimum, so even on this four-site instance a heuristic with no guarantee is not exact, while the FPTAS and the exact program agree. Under the stricter *return reading* (the crew must be back before the fire arrives) the answer changes: at 50 km/h speed all four sites are individually reachable, yet the optimal policy Concow and Magalia are individually infeasible (round trips of 63 and 89 minutes against arrival minutes 52 and 58), although the crew would *still reach* goes straight to Paradise alone rather than visiting Concow, Magalia, or Yankee Hill first: any detour through the smaller communities consumes time Paradise’s deadline cannot absorb. Naive EDD, dispatching earliest-deadline-first regardless, manages to save only Concow’s 710 residents and misses Paradise entirely. Both the exact DP and the FPTAS recover the true optimum them in 31.5 and 44.5 minutes; the optimum is Paradise alone at both speeds. This is why Remark 1 asks the user to state the reading.

Sensitivity to the uncertain inputs. The inputs are uncertain in several ways: the response speed, the arrival minutes (documented ones carry reconstruction error, extrapolated ones more), and the dispatch delay. We draw 5,000 scenarios with speed

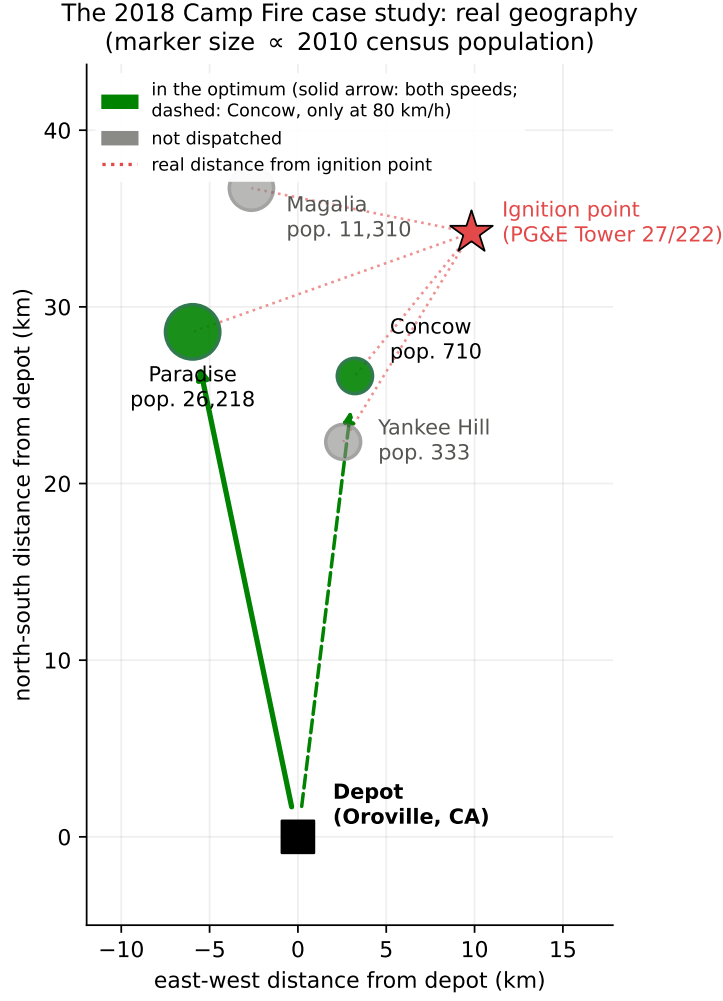


Fig. 5 The real geography behind Table 9: the depot, the fire’s ignition point, and the four communities, plotted from their real coordinates (dotted lines: each community’s real distance from the ignition point, the basis for the documented and extrapolated arrival times). The solid arrow, dispatch to Paradise, is in the optimum at both response speeds; at 80 km/h the optimum also serves Concow, first (dashed arrow).

uniform on $[40, 90]$ km/h, arrival minutes scaled by independent factors uniform on $[0.85, 1.15]$ (documented sites) or $[0.70, 1.30]$ (extrapolated sites), and a departure delay uniform on $[0, 20]$ minutes, and solve each exactly. Paradise is in the optimum in 100% of the draws; the optimal set is Paradise alone in 65.3%, Concow with Paradise in 18.4%, Yankee Hill with Paradise in 13.9% and Magalia with Paradise in every case 2.3%. The plan that is optimal for the nominal 80 km/h instance (Concow, then Paradise) is fully on time in only 20.7% of the draws and retains on average 26.6% of the drawn optimum (fifth percentile 2.7%), because a delay or a slower road lets the

Concow detour make Paradise late. The simpler plan “Paradise only” is on time in all draws and retains 98.6% of the drawn optimum on average (fifth percentile 97.4%). The nominal optimum is therefore brittle, and the greedy repair heuristic matches them here as well. This single real brittleness comes from adding a low-weight site ahead of a dominant one; this is the kind of effect that a model with uncertain parameters should capture (Section 6.3).

This single instance cannot substitute for the synthetic sweepstudy’s statistical evidence, but it shows the same qualitative pattern is not a synthetic data artifact, and the 50 km/h scenario’s already infeasible sites are exactly the kind of situation Section 6.1’s discussion of multiple vehicles is motivated by: a single depot’s single crew has a hard limit on what it can save, regardless of how it is scheduled and its outputs depend on the assumptions above; its role is to show the model, the algorithms, and the reading and uncertainty issues on real geography.

6 Discussion and extensions

The results of Section 4 settle the single-depot, single-vehicle, deterministic-deadline case completely. Three natural directions relax one assumption at a time; we discuss, for each, what carries over from our analysis and what specifically breaks, rather than simply listing them as unexplored future work.

6.1 Multiple depots or vehicles

Suppose $m > 1$ vehicles are available, each able to serve a disjoint subset of sites, still under the same round-trip dispatch-time and deadline structure. Call this serving its own sequence of round trips, as in MWHED- m (MWHED itself is MWHED-1). Two facts are immediate. First, of Section 4.4. Four facts describe what changes.

Hardness persists for every fixed m . Take a common-deadline instance with deadline D and total weight $\Omega = \sum_i w_i$ (the hard instances of Theorem 2) and add $m - 1$ “blocker” sites with $p = d = D$ and weight $\Omega + 1$. A blocker occupies a vehicle for all of $[0, D]$: if anything is served on that vehicle before it, the blocker is late, and if anything is served after it, that site is late. So each served blocker uses a vehicle exclusively, dropping a blocker costs $\Omega + 1$ and can free a vehicle worth at most Ω , and an optimum serves all $m - 1$ blockers and solves the original instance on the one remaining vehicle. Hence MWHED- m is NP-hard for every fixed $m \geq 1$: the $m = 1$ instance embeds into any MWHED- m instance by adding $m - 1$ vehicles with no sites to serve, so Theorem 2’s hardness persists. Second, (Simply adding idle vehicles to an instance would not do: extra vehicles can serve sites, which changes the problem.)

The exact dynamic program grows with m . Theorem 3’s dynamic program does not extend at the same complexity: the natural generalization tracks a completion-time state per vehicle; recursion extends by tracking a completion time per vehicle. Assign sites in deadline order (Lemma 1, applied on each vehicle) and let $f(i, t_1, \dots, t_m)$ be the maximum on-time weight using sites $1, \dots, i$ when vehicle k has accumulated exactly t_k occupancy; site i may be appended to any vehicle k with $t_k \leq d_i$, giving a table of size $O(n \cdot P^m)$ rather than $O(nP)$ —pseudo-polynomial—

$$f(i, t_1, \dots, t_m) = \max \left(f(i-1, t_1, \dots, t_m), \right. \\ \left. \max_{k: p_i \leq t_k \leq d_i} (f(i-1, t_1, \dots, t_k - p_i, \dots, t_m) + w_i) \right).$$

The table has $O(nP^m)$ cells, pseudo-polynomial for fixed m , but with an exponent that grows with the number of vehicles, not merely a constant-factor slowdown. Whether a smarter exact algorithm avoids this blow-up, or whether the m^{th} power is unavoidable, is open; this connects MWHED- m to . Whether this is unavoidable is open here; the parallel-machine scheduling literature's own maximum-weighted-on-time-jobs problem, whose complexity landscape beyond a single machine is treats the problem as considerably less settled than the single-machine case we resolve here (Lenstra et al. 1977). Recent evidence confirms this: Heeger and Molter (2025) show that even the parallel-machine analogue of our tractable equal-cost special case—identical processing times across identical machines, $P|r_j, p_j=p|\sum w_j U_j$ —is already NP-hard, and $W[2]$ -hard parameterized by the number of machines, once release dates are allowed. Translated into our setting, this suggests that Theorem 5's tractability for MWHED-1 is unlikely to survive as cleanly into MWHED- (Lenstra et al. 1977). We conjecture, but have not verified, that value-scaling extends to an m for $m \geq 2$, even restricted to the equal-dispatch-cost case—the multi-vehicle relaxation appears to cost more than just an exponent in the DP's table size. We conjecture the FPTAS of Theorem 4 extends to -dimensional state.

Equal dispatch times stay tractable. Theorem 5 solves MWHED- m for fixed m via the same value-scaling idea applied to an with equal dispatch times in $O(n \log n)$ time for every m -dimensional state (the scaled-weight axis is untouched). This is the multi-vehicle analogue of the single-vehicle special case and holds without release dates.

Release dates break it. Heeger and Molter (2025) show that the unweighted parallel-machine problem with identical processing times and release dates, $P|r_j, p_j=p|\sum U_j$, is NP-hard and $W[2]$ -hard parameterized by the number of vehicles); but we have not verified this in detail and do not claim it as a result here.

To see concretely where the P^m term comes from: assign sites to vehicles greedily by deadline as in Lemma 1, but now track, for each vehicle $k=1, \dots, m$, its own cumulative dispatch time t_k . The natural generalization of $f(i, t)$ in Theorem 3's proof becomes $f(i, t_1, \dots, t_m)$: the maximum on-time weight achievable using sites $1, \dots, i$, where vehicle k has accumulated exactly t_k dispatch time. Site i can be assigned to whichever vehicle k currently has $t_k + p_i \leq d_i$, giving the recursion

$$f(i, t_1, \dots, t_m) = \max \left(f(i-1, t_1, \dots, t_m), \right. \\ \left. \max_{k: t_k \geq p_i, t_k \leq d_i} (f(i-1, t_1, \dots, t_k - p_i, \dots, t_m) + w_i) \right),$$

a direct generalization of (2) with m time axes instead of one. Since each axis ranges over $0, \dots, P$, the table has $O(P^m)$ cells per site, $O(nP^m)$ overall—exactly the blow-up claimed above, and the reason a genuinely new idea, not just a bigger table, would be needed to bring MWHED- m ’s exact algorithm back to a complexity independent of m in the exponent machines. Release dates have no counterpart in MWHED as defined here, since every vehicle is at the depot at time 0 and every site can be served from the start. They would arise if sites become reachable, or crews available, only at given times (for instance, a road cleared by a firebreak). The equal-cost tractability is therefore robust to adding vehicles but not to adding release dates, and extending the model in that direction is a natural open problem.

6.2 A relocating depot or growing-radius hazard

Definition 1 takes each site’s round-trip dispatch time p_i as a fixed constant, appropriate when the depot is stationary. If the depot itself can relocate between dispatches—a mobile command post following a fire line, for instance— p_i becomes a function of the depot’s current position, which in turn depends on the dispatch order already chosen. Lemma 1’s exchange argument relies critically on p_i being order-independent (Section 3’s observation that every visit is a self-contained round trip); with a relocating depot this fails, and the problem acquires genuine routing structure on top of its scheduling structure. We expect this variant to be at least as hard as MWHED (it contains MWHED as the special case of a depot that chooses not to move) and likely strictly harder, since the order in which sites are visited now affects future dispatch costs, not just which deadlines are met.

It is worth being precise about what “growing-radius hazard” does and does not already mean for a *stationary* depot, since the two relaxations named in this subsection’s title are not the same complication. If the hazard simply expands outward from a fixed origin at a fixed, known rate, its arrival time at each site is already fully captured by the deadline d_i in Definition 1—this is exactly the reduction described in Section 1, and requires no extension to our model at all. The genuine second relaxation, distinct from a relocating depot, arises if the spreading hazard itself degrades the road or access network the depot’s vehicle uses—for instance, a wildfire making a route impassable once it crosses that route, or rising floodwater cutting off a bridge—so that p_i is not merely unknown but a function of *when* site i is reached, since a later attempt may have to detour around hazard-affected infrastructure that an earlier attempt would not have. This again breaks Lemma 1’s order-independence assumption, but through the road network’s degradation rather than the depot’s own movement, and the two effects would compound if both are present simultaneously.

6.3 Uncertain parameters

Definition 1 takes ~~each deadline the dispatch times p_i , the deadlines d_i and the weights w_i~~ as known exactly. If d_i is instead a random variable (as it would be if derived from a stochastic hazard-spread simulation), the natural analogue replaces the hard feasibility indicator $\mathbb{1}[C_i(\sigma) \leq d_i]$ in (1) with $\Pr[C_i(\sigma) \leq d_i]$, turning MWHED into a chance-constrained or expected-value scheduling problem. Lemma 1’s proof

does not survive this change unmodified: the exchange argument compares two fixed completion times against two fixed deadlines, and with random deadlines the ordering that maximizes $\sum_s \pi_s \sum_i w_i^s \mathbb{P}[C_i^s(\sigma) \leq d_i^s]$, which for $S = 1$ is MWHED and is therefore NP-hard. Lemma 1 holds scenario by scenario but not simultaneously: no single order sorts all scenarios' deadlines, and the exchange argument fails. A small example makes this concrete: two sites with $p = 1$ each; site A with $w_A = 1$ and a deterministic deadline $d_A = 2$, and site B with $w_B = 10$ and a random deadline $d_B = 1$ or $d_B = 100$, each equal to 1 or 100 with probability $\frac{1}{2}$, so $\mathbb{E}[d_B] = 50.5 > \mathbb{E}[d_A] = 2$. Sorting by increasing $\mathbb{E}[d_i]$ dispatches each, so $\mathbb{E}[d_B] = 50.5 > \mathbb{E}[d_A] = 2$. Serving A first: A completes at $1 \leq 2$ (always on time), and B completes at 2, on time only when $d_B = 100$, giving expected value $1 + 10 \cdot \frac{1}{2} = 6$. Dispatching (the expected-deadline order) gives $1 + 10 \cdot \frac{1}{2} = 6$; serving B first instead: first gives $10 + 1 = 11$, because B completes at 1, on time whenever $d_B \geq 1$, which holds for both possible values of d_B , and A completes at $2 \leq 2$, always on time, giving expected value $10 + 1 = 11$ —nearly double, by visiting the site with the larger expected deadline first, because d_B 's tight realization ($d_B = 1$) is only avoidable by acting immediately; information the expected value alone discards. This would couple MWHED to exactly the kind of sequential decision and statistical monitoring questions studied elsewhere in the stochastic vehicle routing literature, rather than to a direct generalization of Theorems 3–4. This is the setting of two-stage stochastic programming, in which a first-stage plan is evaluated against scenario-dependent recourse; evacuation planning under uncertain demand and network damage (Wang 2020) and facility location under disruptions (Koca 2023) are close relatives in the disaster-response literature. A sample-average formulation of the expected-weight objective is a mixed-integer program whose combinatorial core is MWHED with scenario-dependent deadlines; a Benders-type decomposition by scenario is natural, and exploiting Lemma 1 within a scenario to generate cuts is, we believe, a promising direction that we have not pursued.

Scenario-based (stochastic) models. Let uncertain parameters take values $\xi_s = (p^s, d^s, w^s)$ with probabilities π_s for $s = 1, \dots, S$, and let the dispatch order σ be chosen before the scenario is revealed. The natural objective is the expected on-time weight $\sum_s \pi_s \sum_i w_i^s \mathbb{P}[C_i^s(\sigma) \leq d_i^s]$, which for $S = 1$ is MWHED and is therefore NP-hard. Lemma 1 holds scenario by scenario but not simultaneously: no single order sorts all scenarios' deadlines, and the exchange argument fails. A small example makes this concrete: two sites with $p = 1$ each; site A with $w_A = 1$ and a deterministic deadline $d_A = 2$, and site B with $w_B = 10$ and a random deadline $d_B = 1$ or $d_B = 100$, each equal to 1 or 100 with probability $\frac{1}{2}$, so $\mathbb{E}[d_B] = 50.5 > \mathbb{E}[d_A] = 2$. Sorting by increasing $\mathbb{E}[d_i]$ dispatches each, so $\mathbb{E}[d_B] = 50.5 > \mathbb{E}[d_A] = 2$. Serving A first: A completes at $1 \leq 2$ (always on time), and B completes at 2, on time only when $d_B = 100$, giving expected value $1 + 10 \cdot \frac{1}{2} = 6$. Dispatching (the expected-deadline order) gives $1 + 10 \cdot \frac{1}{2} = 6$; serving B first instead: first gives $10 + 1 = 11$, because B completes at 1, on time whenever $d_B \geq 1$, which holds for both possible values of d_B , and A completes at $2 \leq 2$, always on time, giving expected value $10 + 1 = 11$ —nearly double, by visiting the site with the larger expected deadline first, because d_B 's tight realization ($d_B = 1$) is only avoidable by acting immediately; information the expected value alone discards. This would couple MWHED to exactly the kind of sequential decision and statistical monitoring questions studied elsewhere in the stochastic vehicle routing literature, rather than to a direct generalization of Theorems 3–4. This is the setting of two-stage stochastic programming, in which a first-stage plan is evaluated against scenario-dependent recourse; evacuation planning under uncertain demand and network damage (Wang 2020) and facility location under disruptions (Koca 2023) are close relatives in the disaster-response literature. A sample-average formulation of the expected-weight objective is a mixed-integer program whose combinatorial core is MWHED with scenario-dependent deadlines; a Benders-type decomposition by scenario is natural, and exploiting Lemma 1 within a scenario to generate cuts is, we believe, a promising direction that we have not pursued.

Robust feasibility. Feasibility of a set of sites is monotone in the parameters: it is preserved when d_i increases or p_i decreases. A set that is on time for every realization in a box $[p_i^-, p_i^+] \times [d_i^-, d_i^+]$ is therefore exactly a feasible set of the MWHED

instance (p^+, d^-, w) , and every result above applies to that worst-case instance. This is cheap but conservative; in the case study the worst corner of the sampled ranges (slow roads, early arrival, a 20-minute delay) makes even Paradise individually infeasible, so instance-wise robustness would return nothing. Chance-constrained or scenario-averaged objectives are the useful formulations there.

Calibration and sensitivity. Because the inputs are data-driven, reporting how the optimum moves with them, as in Section 5.6, is part of using the model. The case study suggests a practical rule that scenario models would formalize: when one site dominates the weight, protect it first and add other sites only if they remain feasible under the plausible range of delays.

7 Conclusion

We have ~~given a complete complexity and algorithmic characterization of~~ studied Minimum Weighted Hazard-Exposure Dispatch, a scheduling problem that arises ~~naturally~~ whenever a single depot must choose a dispatch order against a spreading hazard with ~~known~~, site-specific arrival deadlines. The problem is exactly the classical weighted on-time-jobs ~~scheduling problem~~ problem 1 $|| \sum w_j U_j$ in transportation language, ~~and is weakly NP-hard already in its simplest heterogeneous case (;~~ consequently its weak NP-hardness (already at equal deadlines, Theorem 2); an exact ~~by a direct reduction from PARTITION), the~~ pseudo-polynomial dynamic program ~~(Theorem 3) and an FPTAS~~, and the approximation scheme are classical, and we have presented them with complete proofs rather than as new techniques. What we believe the transportation framing adds is a precise account of *which heterogeneity makes dispatch hard*: the problem is polynomial exactly when the dispatch times or the weights are constant and weakly NP-hard otherwise (Theorem 4) ~~give planners exact and provably near-exact tools regardless; and a second, structurally distinct special case—equal dispatch cost, Theorem 5—remains~~ 9), heterogeneous hazard deadlines never cause hardness, and the equal-dispatch-time case remains exactly solvable in ~~near-linear time~~, delineating precisely which combination of network heterogeneity causes the difficulty. These guarantees are not merely convenient: the natural do-nothing fallback of dispatching sites in deadline order with no reconsideration has $O(n \log n)$ time for any number of identical vehicles (Theorem 5), though not once release dates are added. Natural heuristics—deadline order with or without skipping, and a weighted greedy repair—have unbounded worst-case loss relative to the optimum (Proposition 6), so some form of optimization is necessary for any performance guarantee at all, and our numerical illustration confirms this qualitative gap persists ~~ratio~~, so a guarantee requires a rule that reconsiders earlier commitments in light of both weights and dispatch times; the dynamic program and the FPTAS are two such rules, not the only ones. The FPTAS’s analysis is tight to within a factor $1 - \epsilon^2$, although on random instances of realistic size, alongside a weighted Moore-Hodgson-style greedy repair heuristic that—while carrying no proven guarantee—recovers substantially more of the optimum than doing nothing at all ~~it is essentially exact; this is only visible once the instance’s weights are large enough for its rounding to be active.~~

~~Section 6 examines three natural relaxations—multiple depots or vehicles, a relocating depot, and uncertain hazard arrival times—identifying precisely which part of our analysis (the exchange argument, the dynamic program’s state space, or both) each relaxation breaks, rather than leaving them as an unexamined list.~~

~~More broadly, MWHEd is a small, self-contained illustration of a pattern that recurs across combinatorial optimization: a transportation-network reparametrization of~~ The numerical study and the case study carry two lessons beyond the theorems. The reading of the deadline (the crew must arrive before the hazard, or return before it) changes the answer on the same data and must be stated. And the optimum of a model with estimated inputs can be brittle: on the Camp Fire instance the nominal optimum serves a small site ahead of the dominant one and fails under modest delays, whereas protecting the dominant site first is robust. Section 6 discusses how scenario-based and robust formulations would capture this, and which parts of the analysis survive: Lemma 1 holds within a scenario but not across scenarios, which is what makes the stochastic version a genuinely different problem. Natural next steps are a stochastic or chance-constrained MWHEd with a decomposition that uses the earliest-deadline lemma within scenarios, release dates and heterogeneous vehicles, ~~and a decades-old scheduling problem reveals both that the classical hardness and algorithmic machinery transfer over essentially unchanged (Theorems 2–4), and that the specific structural question of which restriction restores tractability (Theorem 5) has a clean, network-interpretable answer—sites equidistant from the depot—that was not visible in the original scheduling formulation’s language of abstract jobs and machines. We see this as the paper’s main methodological contribution alongside its specific results: reframing a classical scheduling question in transportation terms did not just relabel it, it surfaced a distinction (dispatch-cost heterogeneity versus deadline heterogeneity) that determines tractability, and that distinction is exactly what Section 6’s three extensions each perturb in a different way~~relocating depot.

Appendix A Exhaustive verification of the running example

Examples 1–5 rely on Example 1’s instance being small enough to check by exhaustive search, not only by the dynamic program itself. Since Lemma 1 lets us restrict to dispatching each candidate subset S in increasing-deadline order, Table A1 lists all $2^4 = 16$ subsets of Example 1’s four sites ($p = (2, 3, 1, 4)$, $d = (4, 6, 2, 9)$, $w = (5, 8, 3, 10)$), each dispatched in that order, together with its total dispatch time and feasibility. Every feasible subset’s weight is at most 23, and $\{1, 2, 4\}$ attains it, confirming $W^* = 23$ independently of Algorithm 1.

Only the full set $\{1, 2, 3, 4\}$ is infeasible, and it is infeasible for exactly the reason Example 1 describes: dispatching all four sites, even in the best possible (deadline) order, makes site 4 complete one time unit after its deadline. Every other subset is feasible, so the optimization reduces to choosing, among the 15 feasible subsets, the one of largest weight—which Table A1 confirms is $\{1, 2, 4\}$, at $W = 23$, exactly matching Example 1, Example 3’s dynamic-program table, and Example 4’s FPTAS instantiation.

Table A1 All 16 subsets of Example 1’s running instance, dispatched in increasing-deadline order. “Sum p ” is the subset’s total dispatch time; a subset is feasible iff every member completes on time in this order (Lemma 1).

Subset S	Order	Sum p	Feasible?	$W(S)$
$\emptyset$	—	0	yes	0
$\{1\}$	1	2	yes	5
$\{2\}$	2	3	yes	8
$\{3\}$	3	1	yes	3
$\{4\}$	4	4	yes	10
$\{1, 2\}$	1,2	5	yes	13
$\{1, 3\}$	3,1	3	yes	8
$\{1, 4\}$	1,4	6	yes	15
$\{2, 3\}$	3,2	4	yes	11
$\{2, 4\}$	2,4	7	yes	18
$\{3, 4\}$	3,4	5	yes	13
$\{1, 2, 3\}$	3,1,2	6	yes	16
$\{1, 2, 4\}$	1,2,4	9	yes	23
$\{1, 3, 4\}$	3,1,4	7	yes	18
$\{2, 3, 4\}$	3,2,4	8	yes	21
$\{1, 2, 3, 4\}$	3,1,2,4	10	no (site 4 completes at $10 > d_4=9$)	

Appendix B Detailed derivation of the FPTAS’s approximation ratio

Theorem 4’s proof compresses the approximation-ratio derivation into a single chain of inequalities. We spell out each step separately here, for a reader who wants to check the bound without re-deriving it.

Recall the setup: $w_{\max} = \max_i w_i$, $K = \max(1, \epsilon w_{\max}/n)$, $w'_i = \lfloor w_i/K \rfloor$, S^* is the true optimal subset with $W^* = \sum_{i \in S^*} w_i$, and $\hat{S}$ is the subset the FPTAS returns (the feasible subset of largest scaled value v with $g(n, v) < \infty$). As in the main proof, we split on which argument of $K = \max(1, \epsilon w_{\max}/n)$ attains the maximum; the case below covers the case $\epsilon w_{\max}/n < 1$ (where the scaling is vacuous and $\hat{S} = S^*$ exactly) in one line, and spells out the remaining case in full.

Vacuous-scaling case, $\epsilon w_{\max}/n < 1$. Then $K = 1$, so $w'_i = \lfloor w_i \rfloor = w_i$ (the w_i are already integers), the value-indexed dynamic program is identical to Theorem 3’s program, and it returns $\hat{S} = S^*$, so $W(\hat{S}) = W^* \geq (1 - \epsilon)W^*$ immediately, since $\epsilon \in (0, 1)$.

The remaining steps assume $\epsilon w_{\max}/n \geq 1$, so that $K = \epsilon w_{\max}/n$ *exactly* (not merely as an upper bound) — this exact equality, rather than an inequality, is what Step 5 below needs, and is the reason the case split above is necessary rather than cosmetic.

Step 1: relating w_i and w'_i . By definition of the floor function, $w'_i \leq w_i/K < w'_i + 1$, so

$$Kw'_i \leq w_i < K(w'_i + 1). \quad (\text{B1})$$

The right-hand inequality of (B1), rearranged, gives $w_i \leq K(w'_i + 1)$, used in Step 4 below; the left-hand inequality gives $w'_i \leq w_i/K$, used in Step 2.

Step 2: S^* 's scaled value is close to W^*/K . Summing the left-hand inequality of (B1)'s consequence $w'_i \leq w_i/K$ is the wrong direction for a lower bound on $\sum_{i \in S^*} w'_i$, so instead we use $w_i/K - 1 < w'_i$ (from $w_i/K < w'_i + 1$, i.e. the same right-hand inequality of (B1) divided by K and rearranged), summed over S^* :

$$\sum_{i \in S^*} w'_i > \sum_{i \in S^*} \left(\frac{w_i}{K} - 1 \right) = \frac{W^*}{K} - |S^*| \geq \frac{W^*}{K} - n. \quad (\text{B2})$$

Step 3: the FPTAS's returned value is at least as good as S^* 's scaled value. S^* is itself a feasible subset (it is feasible by definition, being the true optimum), so it attains some scaled value $\sum_{i \in S^*} w'_i =: v_0$ with $g(n, v_0) < \infty$. Since $\hat{S}$ is defined as the feasible subset of *largest* scaled value among all v with $g(n, v) < \infty$, its scaled value is at least v_0 :

$$\sum_{i \in \hat{S}} w'_i \geq \sum_{i \in S^*} w'_i \stackrel{(\text{B2})}{>} \frac{W^*}{K} - n. \quad (\text{B3})$$

Step 4: converting $\hat{S}$'s scaled value back to true weight. Using the right-hand inequality of (B1) ($w_i \leq K(w'_i + 1)$) would overcount by Kn if applied termwise and summed; instead we directly lower-bound $\hat{S}$'s true weight using the *left*-hand inequality of (B1) ($Kw'_i \leq w_i$), summed over $\hat{S}$, combined with (B3):

$$W(\hat{S}) = \sum_{i \in \hat{S}} w_i \geq K \sum_{i \in \hat{S}} w'_i \stackrel{(\text{B3})}{>} K \left(\frac{W^*}{K} - n \right) = W^* - Kn. \quad (\text{B4})$$

Step 5: Kn equals $\epsilon w_{\max}$ exactly. In this case $K = \epsilon w_{\max}/n$ exactly (established above), so

$$Kn = \epsilon w_{\max}. \quad (\text{B5})$$

This is an equality, not merely a bound—the reason the case split matters: in the vacuous-scaling case K can instead exceed $\epsilon w_{\max}/n$ (since $K = 1$ there), which is exactly why that case is handled separately rather than folded into this one.

Step 6: bounding $w_{\max}$ by W^* . By Assumption 1, the single highest-weight site is individually feasible ($p_i \leq d_i$), so dispatching it alone is a feasible subset of value $w_{\max}$; since W^* is the optimum over *all* feasible subsets, $w_{\max} \leq W^*$.

Combining Steps 4–6:

$$W(\hat{S}) > W^* - Kn = W^* - \epsilon w_{\max} \geq W^* - \epsilon W^* = (1 - \epsilon)W^*,$$

which gives $W(\hat{S}) > (1 - \epsilon)W^*$ in this case, and $W(\hat{S}) = W^*$ in the vacuous-scaling case; either way, $W(\hat{S}) \geq (1 - \epsilon)W^*$, matching Theorem 4.

Statements and Declarations

Funding

The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Author Contributions

Q.V.D. conceived the problem and developed the theoretical results (the hardness reduction, the exact dynamic program, the FPTAS, the matroid-greedy special case, and the naive-policy lower bound). H.V.V. implemented the algorithms, ran the numerical experiments and the real-world case study, and produced the figures. M.N.D. conducted the literature review and helped position the work relative to prior scheduling and disaster-response research. P.S.N. designed the real-world case study and its practical interpretation. Q.V.D. wrote the first draft of the manuscript, and all authors commented on previous versions, reviewed, and approved the final manuscript.

Data Availability

The synthetic instance generators ~~and all code used to produce every table~~ all algorithms, the exhaustive-search verification, and the scripts that regenerate every table and figure in Section 5 (experiment.py, verify_small.py, case_study_campfire.py, and the scripts in figures/) are included with this submission and are self-contained ~~,-except for- and deterministic given their random seeds.~~ The real-world instance of Section 5.6 ~~'s real-world instance, whose inputs are public-domain-is derived from public~~ figures (a NIST technical report and U.S. Census data, both cited in the text) rather than data collected by the authors; the script that derives ~~the instance from those public-it from those~~ figures and reproduces Tables 9 and 10 ~~and the sensitivity analysis~~ is also included ~~with this submission.~~

Use of Large Language Models

An AI coding assistant (Claude, Anthropic) was used to assist with LaTeX formatting and to run the numerical illustration code reported in Section 5. All theorem statements, proofs, and the manuscript text were authored, verified, and are taken responsibility for by the authors.

References

Avci, M.G., Avci, M., Battarra, M., Erdoğan, G.: The wildfire suppression problem with multiple types of resources. *European Journal of Operational Research* **316**(2), 488–502 (2024) <https://doi.org/10.1016/j.ejor.2024.03.005>

- Alvelos, F.: Dispatching, positioning and routing resources for wildfire initial attack. *Networks* **85**(1), 80–102 (2025) <https://doi.org/10.1002/net.22278>
- Antonov, N., Šůcha, P., Janota, M., Hůla, J.: Minimizing the weighted number of tardy jobs: data-driven heuristic for single-machine scheduling. *Computers & Operations Research* **185**, 107281 (2026) <https://doi.org/10.1016/j.cor.2025.107281>
- Chen, L., Lian, J., Mao, Y., Zhang, G.: A nearly quadratic-time FPTAS for knapsack. *SIAM Journal on Computing* (2025) <https://doi.org/10.1137/24M1686772>
- Delazeri, G., Ritt, M.: Wildfire Suppression: Complexity, Models, and Instances. *arXiv:2603.29865* (2026)
- Garey, M.R., Johnson, D.S.: *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman, San Francisco (1979)
- Gens, G.V., Levner, E.V.: Fast approximation algorithm for job sequencing with deadlines. *Discrete Applied Mathematics* **3**, 313–318 (1981)
- Günzel, M., Brüggem, G., Chen, K.-H., Chen, J.-J.: EDF-like scheduling for self-suspending real-time tasks. In: *2022 IEEE Real-Time Systems Symposium (RTSS)*, pp. 172–184 (2022). <https://doi.org/10.1109/RTSS55097.2022.00024>
- Granda, B., Vitoriano, B., Figueira, J.R.: A mathematical programming approach for a wildfire suppression problem. *Operational Research* **25**(1), 16 (2025) <https://doi.org/10.1007/s12351-024-00882-1>
- Harris, M.G., Forbes, M.A., Taimre, T.: Logic-based Benders decomposition for wildfire suppression. *Computers & Operations Research* **160**, 106392 (2023) <https://doi.org/10.1016/j.cor.2023.106392>
- Heeger, K., Hermelin, D.: Minimizing the weighted number of tardy jobs is $W[1]$ -hard. In: *32nd Annual European Symposium on Algorithms (ESA 2024)* (2024). <https://doi.org/10.4230/LIPIcs.ESA.2024.68>
- Heeger, K., Molter, H.: Minimizing the number of tardy jobs with uniform processing times on parallel machines. In: *42nd International Symposium on Theoretical Aspects of Computer Science (STACS 2025)* (2025). <https://doi.org/10.4230/LIPIcs.STACS.2025.47>
- Hermelin, D., Molter, H., Shabtay, D.: Minimizing the weighted number of tardy jobs via $(\max, +)$ -convolutions. *INFORMS Journal on Computing* **36**(3), 836–848 (2024) <https://doi.org/10.1287/ijoc.2022.0307>
- Hejl, L., Šůcha, P., Novák, A., Hanzálek, Z.: Minimizing the weighted number of tardy jobs on a single machine: Strongly correlated instances. *European Journal of Operational Research* **298**(2), 413–424 (2022) <https://doi.org/10.1016/j.ejor.2021.07.002>

- Ibarra, O.H., Kim, C.E.: Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM* **22**(4), 463–468 (1975) <https://doi.org/10.1145/321906.321909>
- Jin, C.: An improved FPTAS for 0-1 knapsack. In: 46th International Colloquium on Automata, Languages, and Programming (ICALP 2019). *LIPIcs*, vol. 132 (2019). <https://doi.org/10.4230/LIPIcs.ICALP.2019.76>
- Koca, E.: Two-stage stochastic facility location problem with disruptions and restricted shortages. *Computers & Industrial Engineering* **183**, 109484 (2023) <https://doi.org/10.1016/j.cie.2023.109484>
- Korte, B., Vygen, J.: *Combinatorial Optimization: Theory and Algorithms*, 6th edn. *Algorithms and Combinatorics*, vol. 21. Springer, Berlin Heidelberg (2018). <https://doi.org/10.1007/978-3-662-56039-6>
- Liu, C.L., Layland, J.W.: Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM* **20**(1), 46–61 (1973) <https://doi.org/10.1145/321738.321743>
- Lawler, E.L., Moore, J.M.: A functional equation and its application to resource allocation and sequencing problems. *Management Science* **16**(1), 77–84 (1969) <https://doi.org/10.1287/mnsc.16.1.77>
- Lenstra, J.K., Rinnooy Kan, A.H.G., Brucker, P.: Complexity of machine scheduling problems. In: *Annals of Discrete Mathematics* vol. 1, pp. 343–362 (1977). [https://doi.org/10.1016/S0167-5060\(08\)70743-X](https://doi.org/10.1016/S0167-5060(08)70743-X)
- Maranghides, A., Link, E.D., Brown, C.U., Mell, W., Hawks, S., Wilson, M., Brewer, W., Vihnanek, R., Walton, W.D.: A case study of the camp fire – fire progression timeline. NIST Technical Note 2135, National Institute of Standards and Technology (2021). <https://doi.org/10.6028/NIST.TN.2135>
- Moore, J.M.: An n job, one machine sequencing algorithm for minimizing the number of late jobs. *Management Science* **15**(1), 102–109 (1968) <https://doi.org/10.1287/mnsc.15.1.102>
- Ma, Q., Peng, C., Xu, C., Xu, C., Zhang, R.: Multiagent Matroid Upgrading: Greedy is Fair and Efficient. *Proceedings of AAMAS 2026*; arXiv:2606.01309 (2026)
- Rostamian, P., Ahmadi Digebsara, A., Yildirim, K.S., Ardestani-Jaafari, A.: Wild-fire management: A systematic review of optimization under uncertainty and complexity. *Operations Research Forum* **7**(1) (2026) <https://doi.org/10.1007/s43069-026-00604-7>
- Sahni, S.K.: Algorithms for scheduling independent tasks. *Journal of the ACM* **23**(1), 116–127 (1976) <https://doi.org/10.1145/321921.321934>

- Tezcan, B., Eren, T.: Forest fire management and fire suppression strategies: a systematic literature review. *Natural Hazards* **121**(9), 10485–10515 (2025) <https://doi.org/10.1007/s11069-025-07227-x>
- Terao, T.: Faster matroid partition algorithms. *ACM Transactions on Algorithms* **21**(2) (2025) <https://doi.org/10.1145/3707208>
- Wang, L.: A two-stage stochastic programming framework for evacuation planning in disaster responses. *Computers & Industrial Engineering* **145**, 106458 (2020) <https://doi.org/10.1016/j.cie.2020.106458>
- Wang, Y., Lv, B., Zhou, Q., Li, J., Tan, T.: Schedulability analysis for self-suspending tasks under EDF-like scheduling. *IEEE Transactions on Computers* **74**(7), 2364–2375 (2025) <https://doi.org/10.1109/TC.2025.3558079>